ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN



XÂY DỰNG TRỢ LÝ AI CHO HỆ THỐNG THÔNG TIN BỆNH VIỆN

ĐỒ ÁN TỐT NGHIỆP

Chuyên ngành: Công nghệ Thông tin

Giảng viên hướng dẫn: **PGS.TS Nguyễn Hồng Quang**
Sinh viên thực hiện: **Trương Công Thái Đức**
Mã số sinh viên: **20225815**

HÀ NỘI – 2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

1. Mục tiêu và nội dung của đồ án

(a) Mục tiêu: .

(b) Nội dung: .

2. Kết quả đạt được

(a) 1.

(b) 2.

(c) 3.

3. Ý thức làm việc của sinh viên:

Hà Nội, tháng năm 2026

Giảng viên hướng dẫn

PGS.TS Nguyễn Hồng Quang

Lời cảm ơn

Trước hết, em xin được bày tỏ lòng biết ơn sâu sắc tới Thầy/Cô giảng viên hướng dẫn, người đã tận tình chỉ bảo, định hướng khoa học và đồng hành cùng em trong suốt quá trình thực hiện đồ án. Sự hỗ trợ và kinh nghiệm quý báu của Thầy/Cô là nền tảng vững chắc để em có thể hoàn thành công trình này.

Em cũng xin chân thành cảm ơn quý Thầy, Cô trong Khoa Công nghệ Thông tin, Trường Đại học Bách Khoa Hà Nội đã truyền đạt kiến thức, phương pháp nghiên cứu trong suốt những năm học. Những bài giảng và thực hành đã trang bị cho em nền tảng kỹ thuật cần thiết để tiếp cận bài toán phức tạp trong lĩnh vực y tế số.

Em trân trọng cảm ơn các bạn bè và đồng nghiệp đã chia sẻ tài liệu, góp ý, và đồng hành trong những lúc gặp khó khăn. Những buổi thảo luận, phản biện thẳng thắn đã giúp em nhìn nhận vấn đề đa chiều hơn.

Dù đã chuẩn bị kỹ lưỡng trong quá trình thực hiện, nhưng em hiểu rằng đồ án không tránh khỏi những thiếu sót nhất định. Em rất mong nhận được những nhận xét và góp ý quý báu từ các Thầy, Cô để báo cáo được hoàn thiện hơn.

Một lần nữa, em xin chân thành cảm ơn!

Hà Nội, tháng năm 2026

Tác giả Đồ án

Trương Công Thái Đức

Tóm tắt nội dung Đồ án

Đồ án trình bày quá trình xây dựng **HIS Agent** — một trợ lý AI tích hợp vào Hệ thống Thông tin Bệnh viện (Hospital Information System) dành cho Bệnh viện Đa khoa Hà Nội. Hệ thống được thiết kế phục vụ hai nhóm người dùng chính: bệnh nhân và bác sĩ, với giao diện hội thoại tự nhiên bằng tiếng Việt.

Nội dung đồ án được triển khai qua ba phần chính:

1. **Tổng quan và khảo sát:** Khảo sát hiện trạng Hệ thống Thông tin Bệnh viện tại Việt Nam, khảo sát các mô hình ngôn ngữ lớn (LLM) ứng dụng trong y tế, chuẩn HL7 FHIR, và kiến trúc agent với tool calling.
2. **Phân tích và thiết kế hệ thống:** Phân tích yêu cầu chức năng và phi chức năng; thiết kế kiến trúc phân lớp, agent loop, phân quyền theo vai trò, module sàng lọc sốt xuất huyết Dengue, và RAG gợi ý khoa.
3. **Xây dựng hệ thống:** Triển khai đầy đủ các thành phần: 26 công cụ FHIR (bao gồm Clinical Decision Support cho sinh hiệu, dị ứng và kê đơn thuốc), module phân tích CBC, API SSE streaming với FastAPI, giao diện web React + TypeScript, xác thực hai lớp cho cổng bệnh nhân, và gửi email tóm tắt khám tự động.

Hà Nội, tháng năm 2026

Tác giả đồ án

Trương Công Thái Đức

MỤC LỤC

Bảng ký hiệu và chữ viết tắt..... iv

CHƯƠNG 1. Tổng quan và khảo sát 1

1.1 Đặt vấn đề..... 1

1.2 Phạm vi và mục tiêu đồ án..... 2

1.2.1 Phạm vi..... 2

1.2.2 Mục tiêu..... 3

1.3 Khảo sát các giải pháp hiện có 3

1.3.1 Các hệ thống HIS tại Việt Nam 3

1.3.2 Các mô hình LLM ứng dụng trong y tế 4

1.3.3 Kiến trúc Agent với tool calling..... 5

1.3.4 Chuẩn HL7 FHIR 5

1.4 Hướng tiếp cận của đồ án 5

CHƯƠNG 2. Phân tích và thiết kế hệ thống 7

2.1 Phân tích yêu cầu..... 7

2.1.1 Sơ đồ Use Case tổng thể 7

2.1.2 Yêu cầu chức năng 8

2.1.3 Yêu cầu phi chức năng..... 10

2.2 Kiến trúc tổng thể hệ thống 10

2.2.1 Tổng quan kiến trúc..... 10

2.2.2 Luồng xử lý yêu cầu 11

2.3 Thiết kế thành phần nền tảng 12

2.3.1 Định dạng dữ liệu trao đổi: prompt, input người dùng và output 12

2.3.2 Thiết kế Agent Loop..... 16

2.3.3 Thiết kế phân quyền theo vai trò 17

2.3.4 Thiết kế công cụ FHIR 19
2.4 Thiết kế các module nghiệp vụ lâm sàng 21
2.4.1 Thiết kế module sàng lọc SXHD 21
2.4.2 Thiết kế module tra cứu số liệu dịch bệnh 23
2.4.3 Thiết kế RAG cho gợi ý khoa 25
2.4.4 Thiết kế module tra cứu thông tin thuốc (RAG truy hồi qua API) 26
2.4.5 Thiết kế module Sinh hiệu với Clinical Decision Support 27
2.4.6 Thiết kế Drug Catalog và Allergy Management 28
2.4.7 Thiết kế module Kê đơn thuốc với 3-tier Allergy Check 29
2.4.8 Thiết kế module Complete Visit Modal 31
2.5 Thiết kế xác thực và bảo mật người dùng 33
CHƯƠNG 3. Xây dựng hệ thống 37
3.1 Môi trường và công nghệ sử dụng 37
3.2 Cấu trúc thư mục dự án 38
3.3 Thiết kế cơ sở dữ liệu 39
3.3.1 Lớp dữ liệu lâm sàng — Aidbox FHIR R5 39
3.3.2 Lớp xác thực — SQLite 39
3.4 Cài đặt lớp Agent 39
3.4.1 Nạp động công cụ 39
3.4.2 Sơ đồ luồng Agent Loop 40
3.4.3 Hỗ trợ ảnh (Vision) cho phân tích CBC 41
3.5 Cài đặt module sàng lọc SXHD 43
3.5.1 Phân tích CBC (dengue_cbc_predict) 43
3.5.2 Luồng phân tích CBC từ ảnh (Flow E — Bác sĩ) 43
3.6 Cài đặt module tra cứu số liệu dịch bệnh 44
3.6.1 Bóc tách dữ liệu bằng regex hai tầng 44

3.6.2 Tra cứu động và xử lý nhiễu tìm kiếm 45
3.6.3 Cache lịch sử và so sánh tuần-trên-tuần 45
3.7 Cài đặt module tra cứu thông tin thuốc (drug_info) 46
3.7.1 Truy hồi từ NCBI bằng tìm kiếm từ khóa 46
3.7.2 Gắn nhãn nguồn cho bằng chứng — chống lệch trích dẫn 46
3.7.3 Hiển thị thẻ nguồn và chip trích dẫn 47
3.8 Cài đặt module Sinh hiệu, Dị ứng và Đơn thuốc 47
3.8.1 Cài đặt module Sinh hiệu (fhir_create_vitals) 47
3.8.2 Cài đặt module Allergy Check 4 lớp (fhir_check_drug_allergy) 48
3.8.3 Cài đặt module Kê đơn (fhir_create_prescription) 49
3.8.4 Cài đặt Complete Visit Modal và Email tóm tắt 50
3.9 Cài đặt API với FastAPI 50
3.9.1 Endpoint chat streaming 50
3.10 Giao diện người dùng 51
3.10.1 Giao diện tổng quan 51
3.10.2 Kết quả các chức năng phía bệnh nhân 51
3.10.3 Kết quả các chức năng hỗ trợ quyết định lâm sàng (bác sĩ) 51
CHƯƠNG 4. Kiểm thử, đánh giá và triển khai 59
4.1 Kiểm thử hệ thống 59
4.1.1 Đánh giá độ chính xác module sàng lọc SXHD 59
4.1.2 Kiểm thử module phân tích CBC 60
4.1.3 Kiểm thử module Sinh hiệu và cảnh báo CDS 60
4.1.4 Kiểm thử module Drug Allergy Check 4 lớp 61
4.1.5 Kiểm thử module Kê đơn và Email tóm tắt 62
4.1.6 Kiểm thử phân quyền theo vai trò 62
4.1.7 Kiểm thử luồng chức năng 63

4.1.8 Kiểm thử module tra cứu số liệu dịch bệnh 64

4.1.9 Kiểm chứng độ chính xác trích dẫn của module tra cứu thuốc 65

4.1.10 Đo hiệu năng thời gian phản hồi 65

4.2 Triển khai hệ thống 67

Kết luận 68

TÀI LIỆU THAM KHẢO 69

DANH MỤC HÌNH VẼ

Hình 1.1 Số ca mắc sốt xuất huyết Dengue tại Việt Nam giai đoạn 2019–2023 (Nguồn: Cục Y tế dự phòng – Bộ Y tế) 2

Hình 2.1 Sơ đồ Use Case toàn bộ hệ thống HIS Agent (26 use cases, trong đó 9 use case CDS và 4 use case tra cứu tri thức ngoài: thuốc qua NCBI, dịch bệnh qua HCDC) 7
Hình 2.2 Kiến trúc phân lớp hệ thống HIS Agent 10
Hình 2.3 Sơ đồ tuần tự Flow A — Bệnh nhân đặt lịch khám qua hội thoại . 11
Hình 2.4 Sơ đồ tuần tự Flow B — Triage thông minh phân loại mức độ ưu tiên 12
Hình 2.5 Cấu trúc system prompt lắp ghép động theo vai trò: khối đăng nhập (A) + prompt nền (B) + context bổ sung (C) 13
Hình 2.6 Cây quyết định phân loại mức độ SXHD theo QĐ 2760 22
Hình 2.7 Sơ đồ tuần tự Flow D — Sàng lọc SXHD qua hội thoại 2 lượt . . . 22
Hình 2.8 Luồng xử lý truy vấn số liệu dịch bệnh (lazy-refresh + tra cứu động) 24
Hình 2.9 Pipeline RAG gợi ý khoa phù hợp dựa trên triệu chứng 25
Hình 2.10 Sơ đồ tuần tự Flow C — Gợi ý khoa bằng RAG từ mô tả triệu chứng 26
Hình 2.11 Sơ đồ tuần tự 3-tier allergy check khi kê đơn cho BN dị ứng Penicillin 31
Hình 2.12 Sơ đồ tuần tự Complete Visit Modal — 4 hành động atomic khi bác sĩ kết thúc khám 33

Hình 3.1 Cấu trúc bảng SQLite và liên kết với FHIR resource 39
Hình 3.2 Lưu đồ vòng lặp Agent (Tool-Calling Loop) 41
Hình 3.3 Sơ đồ tuần tự Flow E — Bác sĩ phân tích CBC từ ảnh upload (Doctor Only) 44
Hình 3.4 Giao diện hội thoại bệnh nhân 52
Hình 3.5 Giao diện tổng quan dành cho bác sĩ 52
Hình 3.6 Phân loại mức độ ưu tiên (triage) và gợi ý khoa khám bằng RAG (F-P04, F-P05) 53
Hình 3.7 Đặt lịch khám ngay trong hội thoại (F-P01) 53
Hình 3.8 Sàng lọc sốt xuất huyết từ triệu chứng với phân độ cảnh báo theo QĐ 2760 (F-P06) 54
Hình 3.9 Tra cứu thông tin thuốc kèm thẻ nguồn NCBI và chip trích dẫn $[n]$ kiểm chứng được (F-P12) 54
Hình 3.10 Tra cứu số ca sốt xuất huyết thực tế từ HCDC kèm trích dẫn nguồn (F-P13) 55

Hình 3.11 Email tóm tắt lần khám gửi tự động cho bệnh nhân (F-P11) 55
Hình 3.12 Phân tích chỉ số CBC và đánh giá nguy cơ sốt xuất huyết (F-D03) 56
Hình 3.13 Ghi nhận sinh hiệu với cảnh báo bất thường tự động theo NEWS2 và QĐ 3192 (F-D07) 57
Hình 3.14 Cảnh báo dị ứng khi kê đơn — phản ứng chéo Penicillin–Cephalosporin (F-D09) 58

Hình 4.1 Sơ đồ triển khai hệ thống HIS Agent 67

DANH MỤC BẢNG BIỂU

Bảng 1.1 So sánh các hệ thống HIS tiêu biểu với HIS Agent 4
Bảng 1.2 So sánh các mô hình LLM theo tiêu chí ứng dụng y tế 4

Bảng 2.1 Bảng tóm tắt format input/output của 9 tool chính 16
Bảng 2.2 Danh sách 26 công cụ của HIS Agent và phân quyền theo vai trò (8 công cụ mới thuộc nhóm CDS) 20
Bảng 2.3 So sánh hai biến thể RAG trong HIS Agent 27
Bảng 2.4 Bảng 7 cặp cross-reactivity với nguồn open-access đã verify . . . 30

Bảng 3.1 Các FHIR resource sử dụng trong hệ thống 39
Bảng 3.2 Năm mẫu regex trích số liệu từ nội dung bản tin HCDC 45
Bảng 3.3 Sửa lỗi lệch số trích dẫn nhờ gắn nhãn nguồn cho bằng chứng . . . 46

Bảng 4.1 Kết quả kiểm thử đơn vị module sàng lọc SXHD (20 ca) 59
Bảng 4.2 Kết quả kiểm thử end-to-end sàng lọc SXHD qua LLM (15 ca) . . 59
Bảng 4.3 Kết quả kiểm thử module đánh giá nguy cơ SXHD từ CBC 60
Bảng 4.4 5 test case allergy check với BN có dị ứng Penicillin severe 61
Bảng 4.5 Kiểm thử phân quyền theo vai trò 63
Bảng 4.6 Kết quả kiểm thử các luồng chức năng chính 64
Bảng 4.7 Kết quả kiểm thử module tra cứu số liệu dịch bệnh 64
Bảng 4.8 Kết quả kiểm chứng grounding và độ chính xác trích dẫn của drug_info 65
Bảng 4.9 Thời gian phản hồi trung bình theo luồng (đo trên môi trường localhost) 66

Bảng ký hiệu và chữ viết tắt

AI	Artificial Intelligence — Trí tuệ nhân tạo
API	Application Programming Interface — Giao diện lập trình ứng dụng
CBC	Complete Blood Count — Công thức máu toàn phần
CORS	Cross-Origin Resource Sharing
FHIR	Fast Healthcare Interoperability Resources
HCT	Hematocrit
HGB	Hemoglobin
HIS	Hospital Information System — Hệ thống thông tin bệnh viện
HL7	Health Level Seven International
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model — Mô hình ngôn ngữ lớn
NLP	Natural Language Processing — Xử lý ngôn ngữ tự nhiên
PLT	Platelet — Tiểu cầu
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SDK	Software Development Kit
SSE	Server-Sent Events
SXHD	Sốt xuất huyết Dengue
UUID	Universally Unique Identifier
WBC	White Blood Cell — Bạch cầu

CHƯƠNG 1. Tổng quan và khảo sát

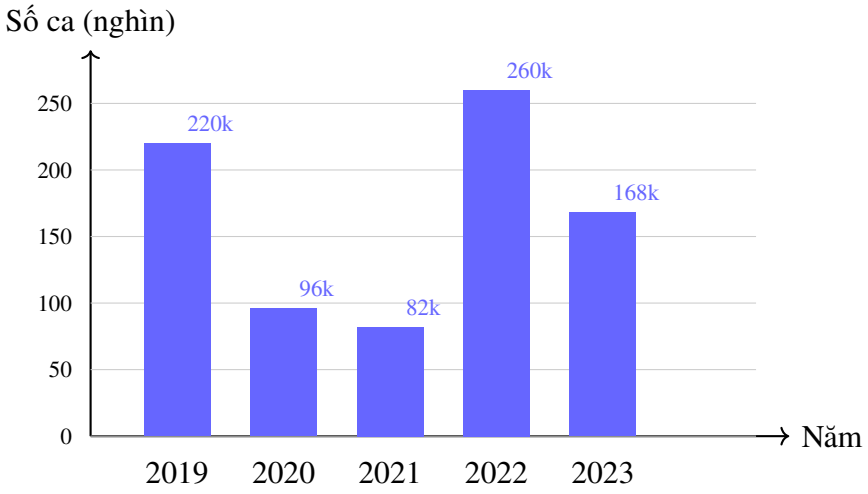
1.1 Đặt vấn đề

Hệ thống thông tin bệnh viện (Hospital Information System — HIS) đóng vai trò trung tâm trong quản lý hoạt động y tế, từ lưu trữ hồ sơ bệnh nhân, theo dõi xét nghiệm, đến quản lý lịch hẹn và thanh toán. Tuy nhiên, phần lớn các hệ thống HIS hiện tại tại Việt Nam vẫn yêu cầu người dùng tương tác qua giao diện form phức tạp, đòi hỏi đào tạo chuyên sâu, và thiếu khả năng hỗ trợ ra quyết định lâm sàng.

Trong bối cảnh bùng nổ của các mô hình ngôn ngữ lớn (Large Language Model — LLM), đặc biệt là các mô hình hỗ trợ tương tác hội thoại tự nhiên, cơ hội xây dựng một lớp trợ lý AI tích hợp vào HIS là rất khả thi. Trợ lý AI có thể giúp:

- Bệnh nhân tự phục vụ (đặt lịch, kiểm tra lịch hẹn) mà không cần gọi điện hay đến quầy lễ tân.
- Triage thông minh: phân loại mức độ ưu tiên dựa trên triệu chứng, gợi ý khoa phù hợp.
- Sàng lọc dịch bệnh (sốt xuất huyết Dengue) theo phác đồ chuẩn của Bộ Y tế.
- Bác sĩ tra cứu hồ sơ, xem kết quả xét nghiệm, phân tích CBC nhanh chóng qua ngôn ngữ tự nhiên.

Việt Nam đang đối mặt với gánh nặng dịch bệnh sốt xuất huyết Dengue (SXHD) tái diễn hàng năm. Theo số liệu của Bộ Y tế, mỗi năm có hàng chục nghìn ca mắc, với nhiều trường hợp nhập viện muộn do nhận biết triệu chứng chưa đúng. Việc tích hợp module sàng lọc SXHD tự động vào giao diện hội thoại của bệnh nhân có thể giúp phát hiện sớm và giảm tải cho cơ sở y tế.



Hình 1.1: Số ca mắc sốt xuất huyết Dengue tại Việt Nam giai đoạn 2019–2023 (Nguồn: Cục Y tế dự phòng – Bộ Y tế)

Hình 1.1 minh họa xu hướng dịch bệnh SXHD trong 5 năm gần đây. Năm 2022 là đỉnh dịch với hơn 370.000 ca mắc và 143 trường hợp tử vong. Sự biến động theo chu kỳ của SXHD đặt ra nhu cầu cấp thiết về công cụ sàng lọc tự động, nhất là trong bối cảnh nguồn lực y tế còn hạn chế.

1.2 Phạm vi và mục tiêu đồ án

1.2.1 Phạm vi

Đồ án tập trung vào việc xây dựng nguyên mẫu (prototype) hệ thống HIS Agent phục vụ bệnh nhân và bác sĩ tại bệnh viện đa khoa quy mô vừa. Hệ thống không thay thế HIS hiện có mà hoạt động như một lớp trợ lý AI phía trên, tương tác với dữ liệu lâm sàng thông qua chuẩn HL7 FHIR R5.

Phạm vi chức năng bao gồm:

- Đặt lịch khám, hủy lịch, xem lịch hẹn cho bệnh nhân.
- Triage thông minh: phân loại mức độ ưu tiên và gợi ý khoa.
- Sàng lọc sốt xuất huyết Dengue theo QĐ 2760/QĐ-BYT 2023.
- Phân tích kết quả xét nghiệm CBC hỗ trợ đánh giá nguy cơ SXHD — chức năng dành riêng cho bác sĩ.
- Tra cứu hồ sơ bệnh nhân, kết quả xét nghiệm cho bác sĩ.
- Quản lý lịch hẹn (duyệt/từ chối) cho bác sĩ.
- Xác thực người dùng theo vai trò (bệnh nhân/bác sĩ).

1.2.2 Mục tiêu

Mục tiêu được chia thành 2 giai đoạn:

Giai đoạn 1 — Core hệ thống:

1. Xây dựng kiến trúc agent có thể mở rộng (extensible), dễ bổ sung công cụ mới.
2. Tích hợp chuẩn FHIR R5 thông qua Aidbox để đảm bảo khả năng tương tác với các hệ thống y tế khác.
3. Triển khai module sàng lọc SXHD dựa trên bộ tiêu chí của Bộ Y tế Việt Nam và WHO.
4. Đảm bảo phân quyền nghiêm ngặt: chức năng phân tích CBC chỉ khả dụng với bác sĩ.
5. Xây dựng giao diện web đơn giản, phản hồi thời gian thực qua SSE.

Giai đoạn 2 — Clinical Decision Support (CDS):

6. Mở rộng hệ thống để đóng vòng lặp lâm sàng đầy đủ: đo sinh hiệu $\rightarrow$ ghi nhận dị ứng $\rightarrow$ kê đơn thuốc $\rightarrow$ hoàn tất khám $\rightarrow$ gửi email cho BN.
7. Triển khai cảnh báo bất thường sinh hiệu theo 3 nguồn quốc tế và Việt Nam: BYT 3192/QĐ-BYT 2010 (huyết áp), NEWS2 RCP UK 2017 (mạch/nhịp thở/SpO2), Harrison's 21st ed (nhiệt độ).
8. Triển khai kiểm tra dị ứng thuốc *four-layer* (direct match, same class, cross-reactivity, clear) với *three-tier alert* (passive banner, active real-time, hard stop) theo pattern Epic SmartChart / Cerner PowerChart.
9. Bảo vệ cổng bệnh nhân hai lớp độc lập (prompt-level + tool-level enforcement) chống prompt injection.
10. Tự động gửi email HTML tóm tắt lần khám cho bệnh nhân qua Gmail SMTP, gồm chẩn đoán, ghi chú, kế hoạch tái khám và bảng đơn thuốc.

1.3 Khảo sát các giải pháp hiện có

1.3.1 Các hệ thống HIS tại Việt Nam

Một số HIS đang triển khai tại Việt Nam bao gồm HIS-EMR của Bệnh viện Bạch Mai [3], hệ thống HIS Phenikaa, và các nền tảng nhập khẩu như Medisoft [5], eHospital [4] [2]. Tuy nhiên, hầu hết các hệ thống này còn thiếu giao diện hội thoại tự nhiên bằng tiếng Việt, chưa tích hợp AI hỗ trợ quyết định lâm sàng (Clinical Decision Support), và chưa áp dụng đầy đủ chuẩn HL7 FHIR.

Bảng 1.1 so sánh các hệ thống HIS tiêu biểu tại Việt Nam với hướng tiếp cận của đồ án:

Tiêu chí	**HIS-EMR Bạch Mai**	**eHospital**	**Medisoft**	**HIS Agent (đồ án)**
Giao diện hội thoại	Không	Không	Không	**Có (Tiếng Việt)**
AI hỗ trợ quyết định	Không	Giới hạn	Không	**Có (LLM + tool)**
Chuẩn HL7 FHIR	Một phần	Một phần	Không	**FHIR R5 đầy đủ**
Sàng lọc SXHD	Không	Không	Không	**Có (QĐ 2760)**
Phân tích CBC tự động	Không	Không	Không	**Có (bác sĩ)**
Đặt lịch tự phục vụ	Giới hạn	Có	Có	**Có (hội thoại)**
Nguồn mở / tùy biến	Không	Không	Không	**Có (Python)**

Bảng 1.1: So sánh các hệ thống HIS tiêu biểu với HIS Agent

1.3.2 Các mô hình LLM ứng dụng trong y tế

Trên thế giới, một số mô hình LLM chuyên biệt cho y tế đã được nghiên cứu:

- **Med-PaLM 2** (Google) [14]: đạt điểm tương đương bác sĩ trong bài kiểm tra USMLE, nhưng chưa hỗ trợ tiếng Việt và chưa tích hợp với FHIR.
- **ClinicalBERT**: mô hình BERT tiền huấn luyện trên ghi chú lâm sàng MIMIC-III, phù hợp phân loại văn bản y tế nhưng không có khả năng hội thoại.
- **GPT-4 / Claude (Anthropic)** [9]: mô hình đa năng có khả năng xử lý ảnh (vision), hội thoại tiếng Việt tốt, hỗ trợ function calling. Phù hợp làm backbone cho hệ thống agent.

Bảng 1.2 so sánh các mô hình LLM dưới góc độ ứng dụng trong hệ thống HIS Agent:

Tiêu chí	**Med-PaLM 2**	**ClinicalBERT**	**GPT-4o**	**Qwen3-VL-Plus**	**Viet-mistral**
Tool calling	Không	Không	Có	**Có**	Giới hạn
Vision (ảnh)	Có	Không	Có	**Có**	Không
Tiếng Việt	Kém	Kém	Tốt	**Tốt**	Tốt
Tốc độ	Chậm	Nhanh	Trung bình	**Nhanh**	Nhanh
Chi phí	Cao	Thấp	Cao	**Thấp**	Miễn phí
On-premise	Không	Có	Không	Không	**Có**

Bảng 1.2: So sánh các mô hình LLM theo tiêu chí ứng dụng y tế

Dựa trên phân tích, **Qwen3-VL-Plus** (Alibaba DashScope) được lựa chọn vì đáp ứng đồng thời: hỗ trợ tool calling, vision API xử lý ảnh xét nghiệm, tiếng Việt tốt, và chi phí phù hợp cho prototype. Hệ thống được thiết kế tương thích OpenAI SDK để dễ dàng chuyển đổi mô hình trong tương lai.

1.3.3 Kiến trúc Agent với tool calling

Mô hình agent với vòng lặp gọi công cụ (tool-calling loop) là hướng tiếp cận được nhiều nghiên cứu áp dụng để xây dựng trợ lý AI có khả năng tương tác với hệ thống bên ngoài. Kiến trúc này gồm:

1. LLM nhận yêu cầu từ người dùng và context hệ thống.
2. LLM quyết định gọi công cụ nào (tool selection).
3. Hệ thống thực thi công cụ và trả kết quả về cho LLM.
4. LLM tổng hợp kết quả và sinh phản hồi tự nhiên.

OpenAI API và các API tương thích (bao gồm Alibaba DashScope, OpenRouter) đều hỗ trợ cơ chế function_calling / tool_use [8], giúp triển khai kiến trúc agent một cách có cấu trúc.

1.3.4 Chuẩn HL7 FHIR

HL7 FHIR (Fast Healthcare Interoperability Resources) là tiêu chuẩn trao đổi dữ liệu y tế hiện đại nhất, được phát triển bởi tổ chức HL7 International [7]. FHIR R5 định nghĩa các resource như Patient, Appointment, Observation, Practitioner theo định dạng JSON/XML, với REST API chuẩn hóa.

Aidbox của Medplum [10] là một nền tảng FHIR server mã nguồn mở, hỗ trợ triển khai cục bộ qua Docker, tuân thủ FHIR R5, cung cấp REST API đầy đủ và công cụ quản trị trực quan.

1.4 Hướng tiếp cận của đồ án

Dựa trên kết quả khảo sát, đồ án lựa chọn hướng tiếp cận xây dựng một trợ lý AI dạng agent tích hợp vào hệ thống thông tin bệnh viện. Cụ thể, hệ thống sử dụng một mô hình ngôn ngữ lớn thương mại (Qwen3-VL-Plus qua Alibaba DashScope) làm backbone, kết hợp kiến trúc agent với vòng lặp gọi công cụ (tool-calling loop), trong đó mỗi công cụ là một module Python độc lập nhằm dễ bảo trì và mở rộng. Toàn bộ dữ liệu lâm sàng được lưu trữ trên Aidbox FHIR R5 để bảo đảm khả năng tương tác theo chuẩn quốc tế. Để gợi ý khoa khám phù hợp với triệu chứng, hệ thống áp dụng kỹ thuật RAG; còn module sàng lọc sốt xuất huyết được triển khai theo phác đồ QĐ 2760/QĐ-BYT 2023 [1] và hướng

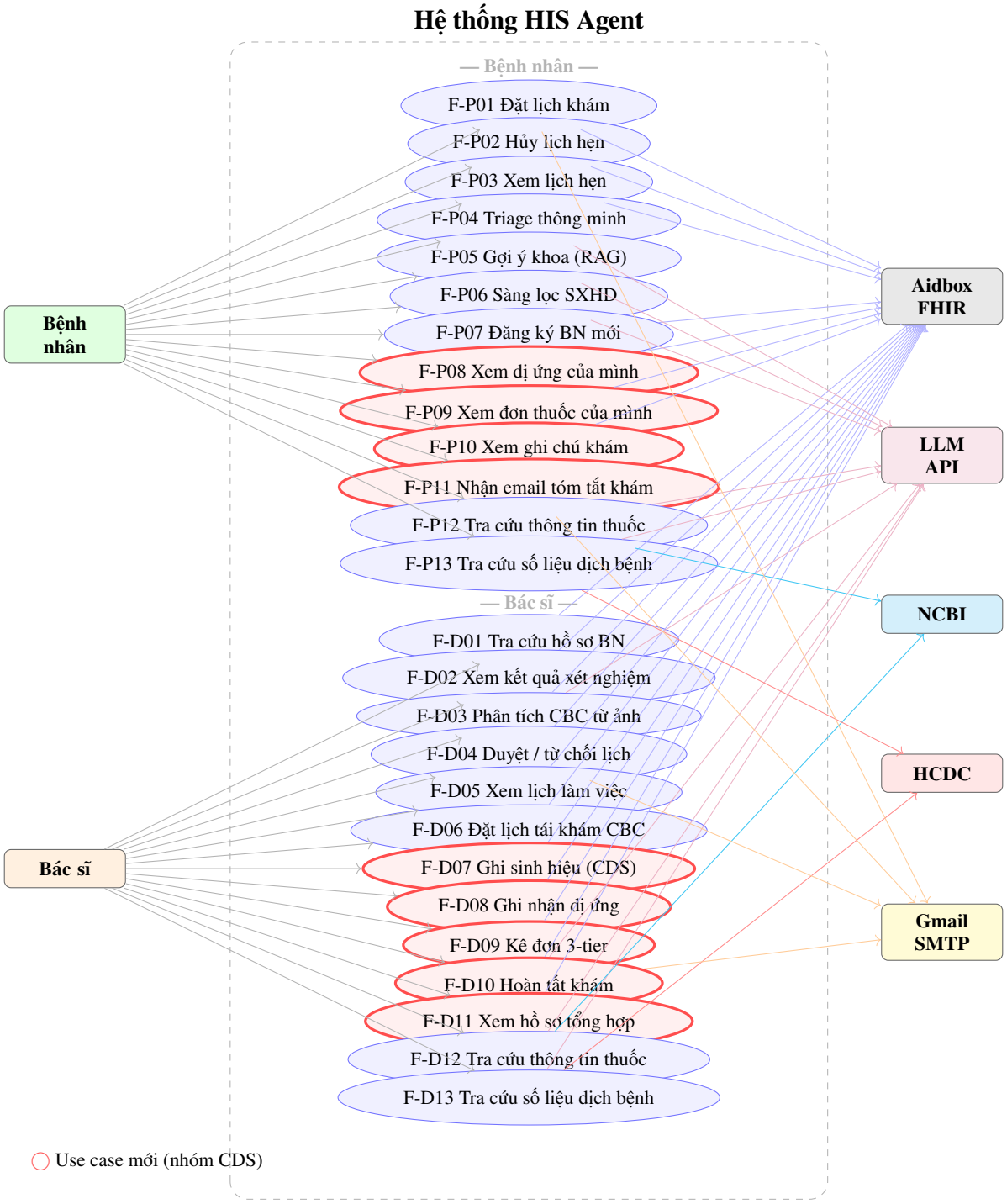
dẫn của WHO 2009 [6]. Cuối cùng, system prompt được phân tách theo vai trò nhằm kiểm soát hành vi của agent phù hợp với từng nhóm người dùng là bệnh nhân và bác sĩ.

CHƯƠNG 2. Phân tích và thiết kế hệ thống

2.1 Phân tích yêu cầu

2.1.1 Sơ đồ Use Case tổng thể

Hình 2.1 trình bày sơ đồ Use Case mô tả toàn bộ chức năng của hệ thống HIS Agent với hai tác nhân chính: Bệnh nhân và Bác sĩ.



Hình 2.1: Sơ đồ Use Case toàn bộ hệ thống HIS Agent (26 use cases, trong đó 9 use case CDS và 4 use case tra cứu tri thức ngoài: thuốc qua NCBI, dịch bệnh qua HCDC)

2.1.2 Yêu cầu chức năng

a, Nhóm người dùng bệnh nhân

ID	Tên chức năng	Mô tả
F-P01	Đặt lịch khám	Bệnh nhân cung cấp triệu chứng hoặc khoa/ngày/giờ, agent đặt lịch và gửi email xác nhận.
F-P02	Hủy lịch hẹn	Bệnh nhân cung cấp tên, agent tra cứu và hủy lịch phù hợp.
F-P03	Xem lịch hẹn	Bệnh nhân hỏi lịch hẹn sắp tới, agent trả về danh sách.
F-P04	Triage thông minh	Agent phân loại mức độ ưu tiên (EMERGENCY / URGENT / SEMI_URGENT / ROUTINE) và gợi ý khoa.
F-P05	Gợi ý khoa	Agent gợi ý khoa phù hợp dựa trên triệu chứng, sử dụng RAG.
F-P06	Sàng lọc SXHD	Agent thực hiện hội thoại 2 lượt để thu thập thông tin, phân loại mức độ SXHD.
F-P07	Đăng ký bệnh nhân mới	Agent thu thập thông tin và tạo hồ sơ bệnh nhân mới trong FHIR.
F-P08	Xem dị ứng của mình	BN hỏi "tôi dị ứng gì"; agent tra cứu FHIR AllergyIntolerance theo patient_id từ session.
F-P09	Xem đơn thuốc của mình	BN hỏi đơn thuốc; agent tra cứu MedicationRequest nhóm theo lần khám.
F-P10	Xem ghi chú khám của bác sĩ	BN xem chẩn đoán + kế hoạch + đơn thuốc trong card tóm tắt lần khám.
F-P11	Nhận email tóm tắt khám	BN nhận email HTML sau khi bác sĩ hoàn tất khám, gồm chẩn đoán + bảng thuốc + cách dùng.
F-P12	Tra cứu thông tin thuốc	BN hỏi liều/tác dụng phụ/tương tác của một thuốc; agent truy hồi NCBI (StatPearls, PubMed), trả lời tiếng Việt kèm thẻ nguồn và chip trích dẫn.
F-P13	Tra cứu số liệu dịch bệnh	BN hỏi số ca mắc ("hiện có bao nhiêu ca SXHD tại TP.HCM"); agent truy hồi số liệu thật theo tuần từ HCDC, trả lời kèm source_url và ghi chú độ trễ/phạm vi.

b, Nhóm người dùng bác sĩ

ID	Tên chức năng	Mô tả
F-D01	Tra cứu hồ sơ bệnh nhân	Tìm kiếm theo tên/BHYT/CCCD, xem lịch sử khám, thuốc, xét nghiệm.
F-D02	Xem kết quả xét nghiệm	Xem kết quả xét nghiệm và sinh hiệu theo danh mục, đánh dấu bất thường.
F-D03	Phân tích CBC	Upload ảnh phiếu xét nghiệm hoặc nhập tay, agent trích xuất chỉ số, lưu FHIR, đánh giá nguy cơ SXHD.
F-D04	Duyệt lịch hẹn	Xem danh sách lịch chờ duyệt, xác nhận hoặc từ chối, gửi email thông báo.
F-D05	Xem lịch khám	Xem lịch làm việc của bác sĩ theo ngày/tuần.
F-D06	Đặt lịch tái khám CBC	Sau kết quả CBC nguy cơ cao, agent gợi ý và đặt lịch tái khám.
F-D07	Ghi nhận sinh hiệu với CDS	Bác sĩ nhập HA/mạch/nhiệt độ/SpO2; hệ thống tự cảnh báo bất thường theo BYT 3192, NEWS2 và Harrison's.
F-D08	Ghi nhận dị ứng	Bác sĩ nhập chất gây dị ứng, mức độ, phản ứng; lưu vào FHIR AllergyIntolerance R5.
F-D09	Kê đơn thuốc với check dị ứng 3 lớp	Bác sĩ chọn thuốc; hệ thống kiểm tra dị ứng theo 4-layer model, hiển thị 3-tier alert (banner, real-time, hard stop).
F-D10	Hoàn tất khám	Một modal đa năng: chẩn đoán + ghi chú + đơn thuốc; lưu ClinicalImpression + MedicationRequest atomic; tự động gửi email cho BN.
F-D11	Xem lịch sử khám tổng hợp	Bác sĩ xem panel BN với sinh hiệu, dị ứng, đơn thuốc, ghi chú lâm sàng.
F-D12	Tra cứu thông tin thuốc	Bác sĩ tra liều/tác dụng phụ/tương tác của thuốc; agent truy hồi NCBI và trả lời có dẫn nguồn, kiểm chứng được.
F-D13	Tra cứu số liệu dịch bệnh	Bác sĩ tra tình hình dịch (số ca SXHD theo tuần tại TP.HCM); agent truy hồi số liệu thật từ HCDC kèm so sánh tuần-trên-tuần và source_url.

2.1.3 Yêu cầu phi chức năng

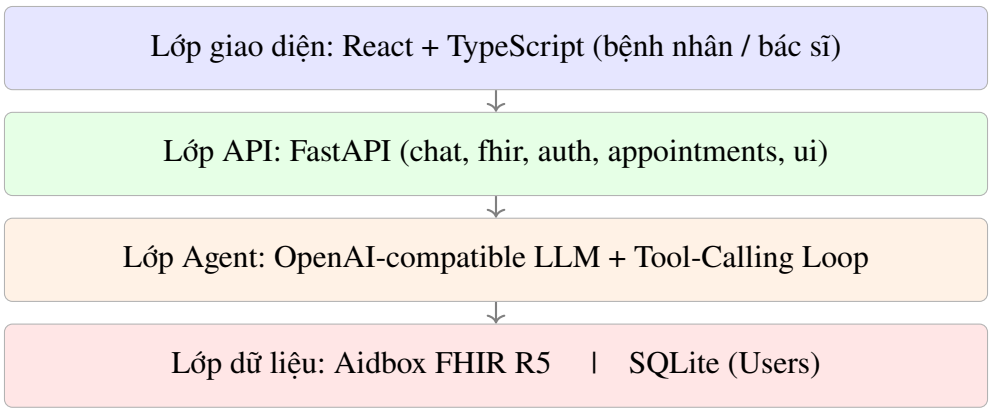
- **Hiệu năng**: Phản hồi đầu tiên trong vòng 2 giây; luồng SSE cung cấp token theo thời gian thực.
- **Bảo mật**: Phân quyền vai trò nghiêm ngặt; chức năng phân tích CBC chỉ dành cho bác sĩ; không hiển thị UUID/ID kỹ thuật ra giao diện bệnh nhân.
- **Độ tin cậy**: Công cụ sàng lọc SXHD dựa trên phác đồ chuẩn, có trích dẫn nguồn rõ ràng; luôn có disclaimer không thay thế chẩn đoán lâm sàng.
- **Khả năng mở rộng**: Kiến trúc module hóa — thêm công cụ mới chỉ cần tạo file Python theo convention, không sửa core.
- **Khả năng tương tác**: Tuân thủ FHIR R5, tương thích với các hệ thống y tế khác.
- **Tiếng Việt**: Toàn bộ giao diện người dùng bằng tiếng Việt, hỗ trợ thuật ngữ y khoa.

2.2 Kiến trúc tổng thể hệ thống

2.2.1 Tổng quan kiến trúc

Hệ thống HIS Agent được tổ chức theo kiến trúc phân lớp (layered architecture) gồm bốn lớp chính:

1. **Lớp giao diện (Presentation Layer)**: Giao diện web React + TypeScript, nhận token thời gian thực qua SSE [15], tách biệt giao diện bệnh nhân và bác sĩ.
2. **Lớp API (Application Layer)**: FastAPI [13] server cung cấp REST endpoints, xử lý xác thực và định tuyến.
3. **Lớp Agent (Business Logic Layer)**: Vòng lặp agent với tool-calling, system prompt phân theo vai trò.
4. **Lớp dữ liệu (Data Layer)**: Aidbox FHIR R5 (dữ liệu lâm sàng) + SQLite (tài khoản người dùng).



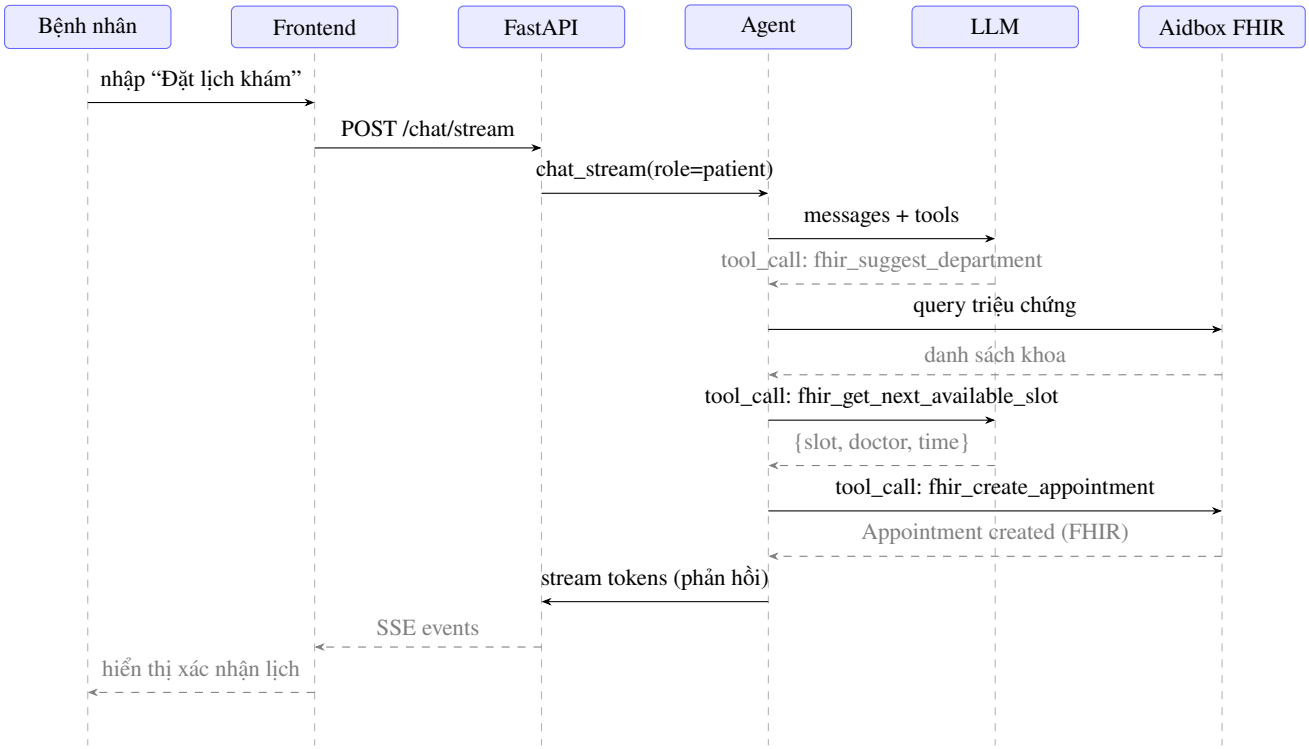
Hình 2.2: Kiến trúc phân lớp hệ thống HIS Agent

2.2.2 Luồng xử lý yêu cầu

Mỗi yêu cầu từ người dùng đi qua các bước:

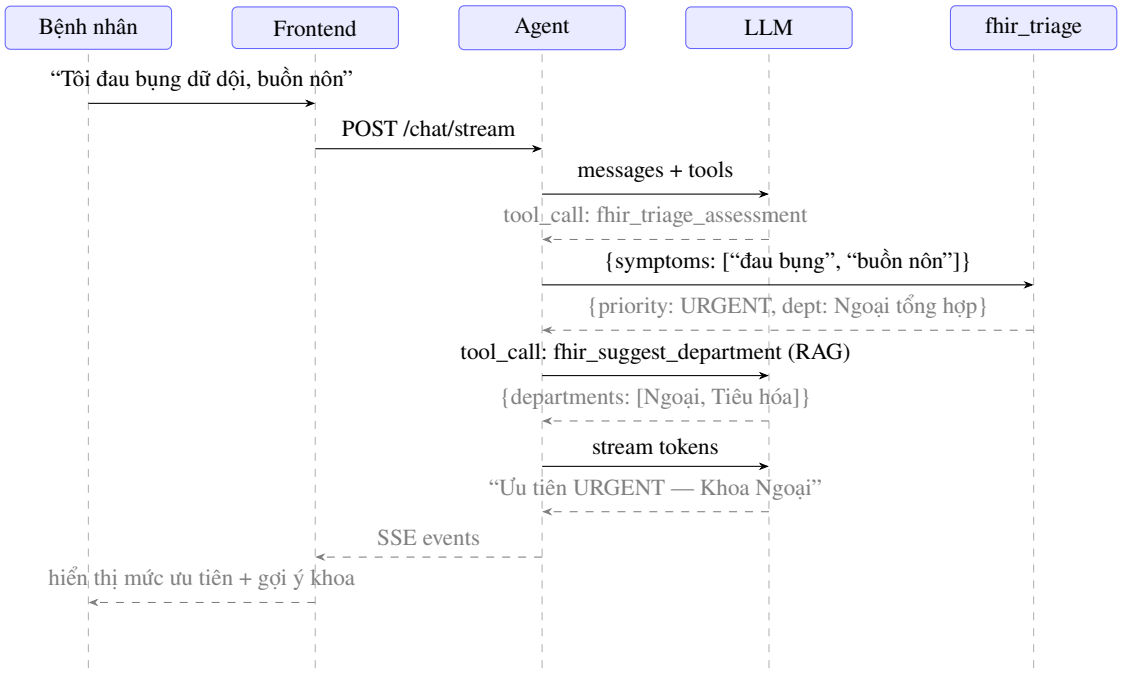
1. Người dùng nhập tin nhắn (và tuỳ chọn ảnh) trên giao diện web.
2. Frontend gửi POST đến /chat/stream với payload gồm message, role, session_id, image_base64.
3. FastAPI load lịch sử hội thoại, gọi chat_stream().
4. chat_stream() chọn system prompt và bộ công cụ tương ứng với role.
5. LLM phân tích yêu cầu và quyết định gọi công cụ (nếu cần).
6. Hệ thống thực thi công cụ (gọi FHIR API hoặc tính toán cục bộ).
7. LLM tổng hợp kết quả, sinh phản hồi bằng tiếng Việt.
8. Phản hồi được stream về frontend qua SSE, hiển thị theo thời gian thực.

Hình 2.3 trình bày sơ đồ tuần tự (Sequence Diagram) cho Flow A — luồng đặt lịch khám trực tiếp của bệnh nhân.



Hình 2.3: Sơ đồ tuần tự Flow A — Bệnh nhân đặt lịch khám qua hội thoại

Hình 2.4 mô tả Flow B — luồng triage thông minh khi bệnh nhân mô tả triệu chứng không rõ khoa:



Hình 2.4: Sơ đồ tuần tự Flow B — Triage thông minh phân loại mức độ ưu tiên

2.3 Thiết kế thành phần nền tảng

2.3.1 Định dạng dữ liệu trao đổi: prompt, input người dùng và output

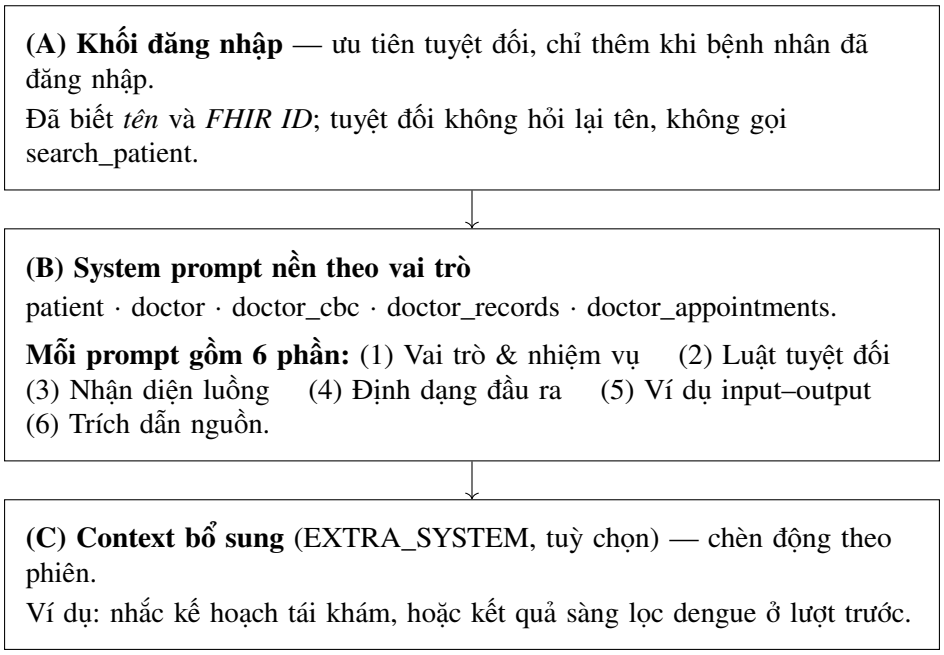
Một trong những điểm cốt lõi đảm bảo tính ổn định và an toàn của hệ thống AI agent là việc chuẩn hóa định dạng cho 3 thành phần: *system prompt* (chỉ dẫn cho LLM), *input người dùng* (câu hỏi, tin nhắn, ảnh đính kèm), và *output trả về* (phản hồi cho người dùng + tool call). Phần này mô tả chi tiết cấu trúc của từng thành phần.

a, Cấu trúc System Prompt

Mỗi role có một system prompt riêng, được build động (*dynamic prompt construction*) ngay khi mỗi request đến. Hình 2.5 mô tả cấu trúc lắp ghép gồm ba khối xếp chồng theo thứ tự.

Cấu trúc mỗi base prompt. Thay vì một đoạn mô tả tự do, mỗi prompt được tổ chức thành sáu phần có tiêu đề rõ ràng. Lý do của cách chia này là một ràng buộc thực tế: hệ thống dùng mô hình ngôn ngữ tương đối nhẹ (để tối ưu chi phí và độ trễ), nên prompt càng có cấu trúc và mệnh lệnh tường minh thì mô hình càng tuân thủ ổn định. Sáu phần, lấy đúng theo các tiêu đề trong prompt thật của vai trò patient, gồm:

1. **Vai trò & nhiệm vụ**: câu mở đầu định danh agent và giới hạn phạm vi, trích nguyên văn: *"Bạn là trợ lý AI của Bệnh viện Đa khoa. Nhiệm vụ: đặt lịch khám, triage triệu chứng, tra cứu thông tin bệnh nhân."* Việc giới hạn nhiệm vụ ngay từ đầu giúp mô hình từ chối yêu cầu ngoài phạm vi.



Hình 2.5: Cấu trúc system prompt lắp ghép động theo vai trò: khối đăng nhập (A) + prompt nền (B) + context bổ sung (C)

2. **Luật tuyệt đối** (mục LUẬT TUYỆT ĐỐI — ĐỌC TRƯỚC KHI LÀM GÌ): các quy tắc an toàn, phần lớn ở dạng phủ định, ví dụ *"KHÔNG hiển thị ID/UUID cho user"*, *"KHÔNG tự điền email"*, *"KHÔNG thực hiện chức năng dành cho bác sĩ"*. Đặt ngay sau phần vai trò để mô hình ưu tiên tuân theo *trước* mọi luồng nghiệp vụ — đây là tuyến phòng thủ an toàn/bảo mật ở tầng prompt.
3. **Nhận diện luồng** (mục NHẬN DIỆN LUỒNG ĐẦU VÀO): bảng định tuyến từ *tín hiệu trong câu người dùng* sang flow xử lý. Ví dụ: "chỉ có triệu chứng" → FLOW B (gọi fhir_triage_assessment ngay); "hỏi nên khám khoa nào" → FLOW C (RAG gợi ý khoa); "nhắc tới sốt xuất huyết" → FLOW D (sàng lọc SXHD). Định tuyến tường minh giúp hành vi agent nhất quán thay vì để mô hình tự suy đoán.
4. **Định dạng đầu ra**: khuôn trình bày cho từng loại kết quả. Ví dụ khối số liệu dịch bệnh phải theo đúng mẫu cố định và *render nguyên văn* trường xu hướng do công cụ trả về, không được tự thêm "tăng/giảm" — nhằm tránh mô hình suy diễn sai số liệu.
5. **Ví dụ input–output**: một vài cặp ví dụ mẫu (*few-shot*) để mô hình bắt chước văn phong và cách quyết định, giảm sai lệch so với chỉ mô tả bằng lời.
6. **Trích dẫn nguồn**: yêu cầu đính kèm nguồn lấy từ trường _citation mà công cụ trả về, bảo đảm mọi khẳng định y khoa đều kiểm chứng được.

Hai quyết định thiết kế quan trọng nhất là: (i) **đặt luật phủ định lên đầu** để ràng buộc an toàn được ưu tiên hơn nghiệp vụ; và (ii) **khối đăng nhập (A) được prepend trước cả base prompt** khi bệnh nhân đã đăng nhập, nhằm *ghi đè* mọi hướng dẫn phía dưới có thể

yêu cầu hỏi lại tên — vừa tránh hỏi thông tin đã biết, vừa chặn rủi ro lộ dữ liệu của bệnh nhân khác. Trích đoạn prompt thật của hai vai trò điển hình được trình bày ở phần sau (xem các đoạn mã role=patient và role=doctor_cbc).

Build prompt động. Mỗi request, hệ thống thực hiện:

```
def build_system(role, patient_id, patient_name,
                 doctor_id, doctor_name, extra_system):
    # 1. Chon base prompt theo role (chen ten/ID bac si neu can)
    if role in DOCTOR_PROMPTS:
        system = DOCTOR_PROMPTS[role].format(
            doctor_name=doctor_name, doctor_id=doctor_id)
    else:
        system = SYSTEM_PROMPT
        # 2. BN da dang nhap -> PREPEND khoi "uu tien tuyet doi"
        if patient_id:
            system = build_login_block(patient_name, patient_id) + s
    # 3. APPEND context dong cua phien (neu co)
    if extra_system:
        system += extra_system
    return system
```

Đoạn mã: Pipeline build system prompt cho mỗi request.

b, Định dạng Input người dùng

Input có 2 loại: **tin nhắn text** và **multimodal** (text + ảnh).

Input dạng text thuần. Frontend gửi HTTP POST /api/chat (kèm header X-Session-Token) với body JSON gồm: role, các trường định danh (patient_id, patient_name, doctor_id, doctor_name), mảng messages chứa lịch sử hội thoại, và extra_system tuỳ chọn. Server-side, hàm _resolve_patient() lấy patient_id thật từ session token, ghi đè các trường định danh rỗng do frontend gửi.

Input multimodal (vision). Khi bác sĩ đính kèm ảnh phiếu CBC, frontend chuyển ảnh sang base64 và bọc trong mảng *content* kiểu OpenAI gồm một phần tử văn bản và một phần tử ảnh (image_url). LLM (Qwen3-VL) đọc cả hai phần: OCR/trích xuất giá trị từ ảnh rồi gọi tool fhir_create_cbc với các giá trị đã đọc.

c, Định dạng Output trả về

Output từ LLM có 2 dạng song song trong cùng một response: *tool_calls* (LLM gọi tool) và *content* (text trả lời user).

Output dạng Tool Call. Khi LLM quyết định gọi tool, response có trường content rỗng và một mảng tool_calls; mỗi phần tử gồm id, tên hàm và arguments. Lưu ý arguments là một *JSON string* (không phải dict), nên agent.py phải gọi json.loads() trước khi truyền vào tool.

Sau khi tool execute, kết quả được bọc thành tin nhắn role=tool (trường content là JSON đã serialize, kèm tool_call_id khớp với lời gọi gốc và trường _citation ghi nguồn dữ liệu) rồi gửi lại LLM. LLM nhận kết quả này có thể gọi tiếp tool khác (vd fhir_create_appointment) hoặc trả lời người dùng.

Output dạng Text (cho user). Khi đã đủ thông tin, response cuối cùng có trường content là văn bản Markdown (tiêu đề in đậm, xuống dòng, và dòng nguồn *"Theo: ..."* ở cuối). Frontend render Markdown này thành HTML và đặt dòng nguồn in nghiêng nhỏ ở cuối bong bóng chat.

d, Định dạng phản hồi qua Server-Sent Events (streaming)

Để cải thiện trải nghiệm, hệ thống dùng *Server-Sent Events* thay vì trả toàn bộ response một lượt: backend phát các sự kiện nối tiếp — token (từng đoạn văn bản), tool (báo đang gọi công cụ) và done (kết thúc). Frontend parse từng sự kiện, nối văn bản vào bong bóng chat theo thời gian thực và hiển thị badge "Đang gọi tool: ..." khi nhận sự kiện tool.

e, Định dạng dữ liệu lâm sàng trên FHIR

Tất cả tool nhận và trả về dữ liệu theo chuẩn FHIR R5 đã trình bày ở các phần trước. Để tham chiếu nhanh, bảng 2.1 tóm tắt input/output của các tool chính:

Tool	Input chính	Output chính
fhir_search_ patient	query (tên/SĐT/BHYT)	{patient_id, name, gender, age, birth_date}
fhir_get_next_ available_slot	department, date_from, practitioner_id (optional)	{found, slot_id, date, time, practitioner, confirm_message}
fhir_create_ appointment	patient_id, department, date, time, patient_email, reason	{success, booking_code, queue_number, practitioner, message}
fhir_triage_ assessment	symptoms (text), known_conditions	{level, label, action, source}
fhir_create_vitals	patient_id, systolic, diastolic, heart_rate, temp, spo2, ...	{saved, observation_id, vitals_saved, alerts[], summary}
fhir_create_allergy	patient_id, substance, category, severity, reaction	{saved, allergy_id, substance, severity_vi, criticality}
fhir_check_drug_ allergy	patient_id, drug_id (hoặc drug_name)	{verdict, checks[], suggested_alternatives[], patient_allergies[]}
fhir_create_ prescription	patient_id, practitioner_id, diagnosis, medications[]	{saved, prescription_id, medications[], warnings[], errors[]}
fhir_send_visit_ summary	patient_email, diagnosis, medications[], practitioner_name	{success, sent_to, subject, med_count}

Bảng 2.1: Bảng tóm tắt format input/output của 9 tool chính

Mọi output đều có trường _citation (string) chứa nguồn dữ liệu, được LLM dùng để render dòng *"□ Theo: ..."* ở cuối câu trả lời. Nếu tool gặp lỗi, output có trường error (string) thay cho các trường thành công — LLM được hướng dẫn (trong system prompt) là phải xin lỗi và đề xuất hành động thay thế.

2.3.2 Thiết kế Agent Loop

Agent loop là thành phần cốt lõi, thực thi vòng lặp gọi công cụ cho đến khi LLM trả về phản hồi cuối:

Vòng lặp lặp lại cho tới khi LLM không còn yêu cầu gọi công cụ; mỗi vòng tương ứng một bước dùng công cụ — đây chính là cơ chế *agent*:

```
def chat_stream(message, history, role, ...):
    system = _select_system_prompt(role)   # prompt + bo cong cu the
    tools  = _get_tools_for_role(role)
    history.append({"role": "user", "content": message})

    while True:                            # lap den khi LLM het goi
        response = llm.create(model=MODEL, tools=tools,
            messages=[{"role":"system","content":system}] + history)

        if response.finish_reason == "tool_calls":
            for tc in response.tool_calls:
                yield sse({"type":"tool_call", "label": LABELS[tc.na
                result = execute_tool(tc.name, tc.args)   # vd: quer
                history.append(tool_result(tc.id, result))
        else:                              # LLM tra loi -
> stream text
            for token in llm.stream(history, tools=None):
                yield sse({"type":"text", "delta": token})
            return
```

Đoạn mã: Thuật toán tool-calling loop (rút gọn).

2.3.3 Thiết kế phân quyền theo vai trò

Hệ thống có 5 vai trò (role), mỗi vai trò tương ứng với một system prompt và tập công cụ:

Role	**Tập công cụ được phép**
patient	Triage, gợi ý khoa, tìm BN, đặt/hủy/xem lịch, sàng lọc SXHD
doctor	Toàn bộ công cụ (không giới hạn)
doctor_records	Tìm BN, xem hồ sơ/xét nghiệm/lịch hẹn, lịch bác sĩ
doctor_cbc	Tìm BN, tạo/xem CBC, đặt lịch
doctor_ appointments	Xem/xác nhận/từ chối lịch chờ, lịch bác sĩ

Việc phân nhỏ vai trò bác sĩ thành các sub-role giúp giảm "nhiễu" trong danh sách công cụ mà LLM nhìn thấy, từ đó tăng độ chính xác trong việc lựa chọn công cụ phù hợp.

Mỗi role ánh xạ đến một system prompt riêng biệt, kiểm soát hành vi của LLM ngay từ đầu. Hai trích đoạn dưới đây minh họa sự khác biệt cốt lõi giữa hai system prompt điển hình (vai trò patient và doctor_cbc):

> **Vai trò.** Bạn là trợ lý AI của Bệnh viện Đa khoa. Nhiệm vụ: đặt lịch khám, triage triệu chứng, sàng lọc SXHD.
>
> **LUẬT TUYỆT ĐỐI:**
> 1. KHÔNG hiển thị ID, UUID, mã kỹ thuật cho người dùng.
> 2. KHÔNG tự điền email — phải hỏi trước khi gọi fhir_create_appointment.
> 3. KHÔNG thực hiện chức năng dành cho bác sĩ (dengue_cbc_predict không có trong toolset bệnh nhân).
>
> **NHẬN DIỆN LUỒNG:**
> FLOW A — đặt lịch (biết đủ khoa + ngày + tên) → book luôn
> FLOW B — triage (chỉ có triệu chứng) → gọi fhir_triage_assessment ngay
> FLOW C — gợi ý khoa → fhir_suggest_department (RAG)
> FLOW D — sàng lọc SXHD → hội thoại 2 lượt *(. . .)*

Trích đoạn: System prompt của vai trò patient (rút gọn).

> **Vai trò.** Bạn là trợ lý phân tích CBC của Bệnh viện Đa khoa, làm việc với {doctor_name} ({doctor_id}).
>
> **Nhiệm vụ DUY NHẤT:** phân tích CBC, lưu FHIR, đánh giá SXHD.
>
> **QUY TẮC:**
> – Không hỏi thêm nếu đã có PLT — đủ để lưu và phân tích.
> – TUYỆT ĐỐI KHÔNG gọi dengue_cbc_predict trực tiếp; phải qua fhir_create_cbc hoặc fhir_get_cbc trước.
> – Trích dẫn nguồn: QĐ 2760/QĐ-BYT 2023 (WHO 2009).
>
> **3 LUỒNG:**
> LUỒNG A — upload ảnh phiếu: đọc ảnh, trích xuất PLT/WBC/HCT/HGB/AST/ALT → fhir_search_patient → fhir_create_cbc
> LUỒNG B — nhập tay (vd “PLT 85”)
> LUỒNG C — xem CBC có sẵn của một bệnh nhân *(. . .)*

Trích đoạn: System prompt của vai trò doctor_cbc (rút gọn).

```
# agent.py -- lua chon system prompt theo role
PROMPTS = {
    "patient":              SYSTEM_PROMPT,
    "doctor":               SYSTEM_PROMPT_DOCTOR,
    "doctor_records":       SYSTEM_PROMPT_DOCTOR_RECORDS,
    "doctor_cbc":           SYSTEM_PROMPT_DOCTOR_CBC,
    "doctor_appointments":  SYSTEM_PROMPT_DOCTOR_APPOINTMENTS,
}
system = PROMPTS.get(role, SYSTEM_PROMPT).format(
    doctor_name=doctor_name or "",
    doctor_id=doctor_id or "",
)
tools = _get_tools_for_role(role)
```

Đoạn mã: Ánh xạ role sang system prompt trong agent.py.

Sự khác biệt then chốt giữa hai prompt: (1) role patient cấm tường minh mọi chức năng lâm sàng và không có dengue_cbc_predict trong toolset; (2) role doctor_cbc nhận doctor_name và doctor_id qua f-string, có quyền truy cập đầy đủ CBC tools, và quy định thứ tự gọi tool nghiêm ngặt để tránh gọi trực tiếp dengue_cbc_predict mà không lưu FHIR trước.

2.3.4 Thiết kế công cụ FHIR

Mỗi công cụ FHIR là một module Python độc lập trong thư mục tools/fhir/, tuân theo convention:

```
TOOL_NAME    = "fhir_example"
DESCRIPTION  = "Mo ta cong cu"
ARGS_SCHEMA  = {
    "patient_id": "string -- FHIR Patient ID",
}
def run(args: dict) -> dict:
    ...
    return {"result": ...}
```

Đoạn mã: Convention định nghĩa công cụ FHIR.

Bảng 2.2 liệt kê đầy đủ 26 công cụ của hệ thống (18 công cụ ban đầu và 8 công cụ mở rộng phục vụ Clinical Decision Support), mô tả chức năng và phân quyền theo vai trò:

Bảng 2.2: Danh sách 26 công cụ của HIS Agent và phân quyền theo vai trò (8 công cụ mới thuộc nhóm CDS)

Tên công cụ	**Mô tả**	**Role được phép**
fhir_search_patient	Tìm bệnh nhân theo tên, SĐT, BHYT/CCCD	patient, doctor, doctor_*
fhir_triage_assessment	Phân loại triệu chứng, xác định mức ưu tiên khám	patient, doctor
fhir_suggest_department	Gợi ý khoa khám từ triệu chứng (RAG MiniLM)	patient, doctor
fhir_get_next_available_slot	Tìm khung giờ khám trống gần nhất	patient, doctor
fhir_create_appointment	Tạo lịch hẹn khám mới trong FHIR	patient, doctor
fhir_cancel_appointment	Hủy lịch hẹn đang chờ xác nhận	patient, doctor
fhir_get_patient_appointments	Xem danh sách lịch hẹn của bệnh nhân	patient, doctor, doctor_records
fhir_get_patient_history	Lấy lịch sử khám bệnh (DiagnosticReport)	doctor, doctor_records
fhir_get_observations	Lấy kết quả xét nghiệm (Observation)	doctor, doctor_records
fhir_create_patient	Tạo hồ sơ bệnh nhân mới trong Aidbox FHIR	doctor
fhir_create_cbc	Lưu kết quả CBC vào FHIR (Observation)	doctor, doctor_cbc
fhir_get_cbc	Truy vấn kết quả CBC đã lưu	doctor, doctor_cbc, doctor_records
fhir_confirm_appointment	Xác nhận lịch hẹn đang chờ duyệt	doctor, doctor_appointments
fhir_reject_appointment	Từ chối lịch hẹn đang chờ duyệt	doctor, doctor_appointments
fhir_get_pending_appointments	Lấy danh sách lịch hẹn chờ xác nhận	doctor, doctor_appointments
fhir_get_doctor_schedule	Xem lịch làm việc của bác sĩ	doctor, doctor_records, doctor_appointments

Tên công cụ	Mô tả	Role được phép
dengue_symptom_ assessment	Sàng lọc SXHD theo triệu chứng (WHO/QĐ 2760)	patient, doctor
dengue_cbc_predict	Đánh giá nguy cơ SXHD từ CBC — **chỉ bác sĩ**	doctor, doctor_cbc
fhir_create_vitals	Lưu sinh hiệu (HA, mạch, nhiệt độ, SpO2...) với cảnh báo tự động	doctor
fhir_create_allergy	Ghi nhận dị ứng (thuốc/thực phẩm) theo FHIR R5 Allergy-Intolerance	doctor
fhir_get_allergies	Lấy danh sách dị ứng của bệnh nhân	patient (own), doctor
fhir_check_drug_ allergy	Kiểm tra dị ứng 4 lớp trước khi kê thuốc	doctor
fhir_create_ prescription	Tạo đơn thuốc nhiều thuốc (MedicationRequest)	doctor
fhir_get_prescriptions	Lấy lịch sử đơn thuốc theo nhóm	patient (own), doctor
fhir_get_clinical_ impressions	Lấy ghi chú lâm sàng (ClinicalImpression)	patient (own), doctor
fhir_send_visit_ summary	Gửi email tóm tắt lần khám qua Gmail SMTP	doctor

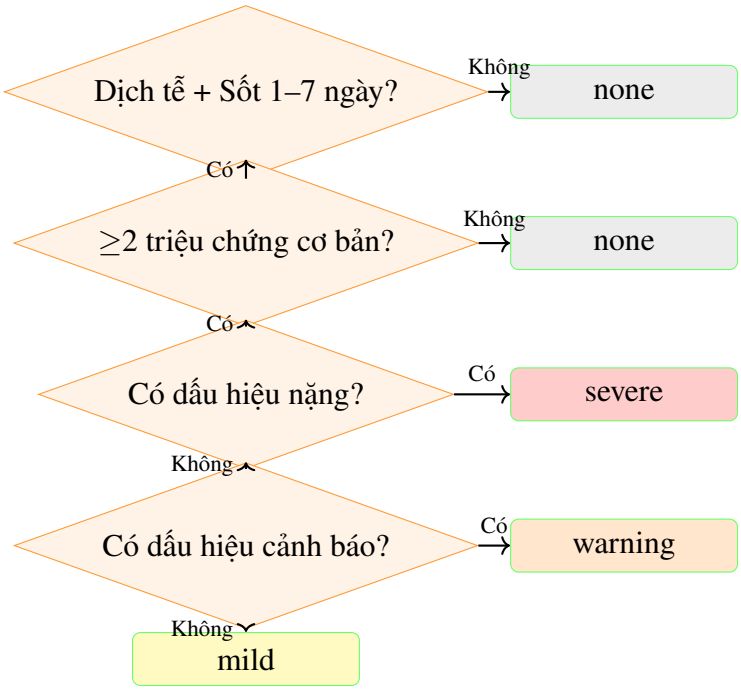
2.4 Thiết kế các module nghiệp vụ lâm sàng

Phần này trình bày thiết kế các module trực tiếp phục vụ chuyên môn lâm sàng: sàng lọc bệnh, truy hồi tri thức y khoa và hỗ trợ quyết định lâm sàng cho bác sĩ.

2.4.1 Thiết kế module sàng lọc SXHD

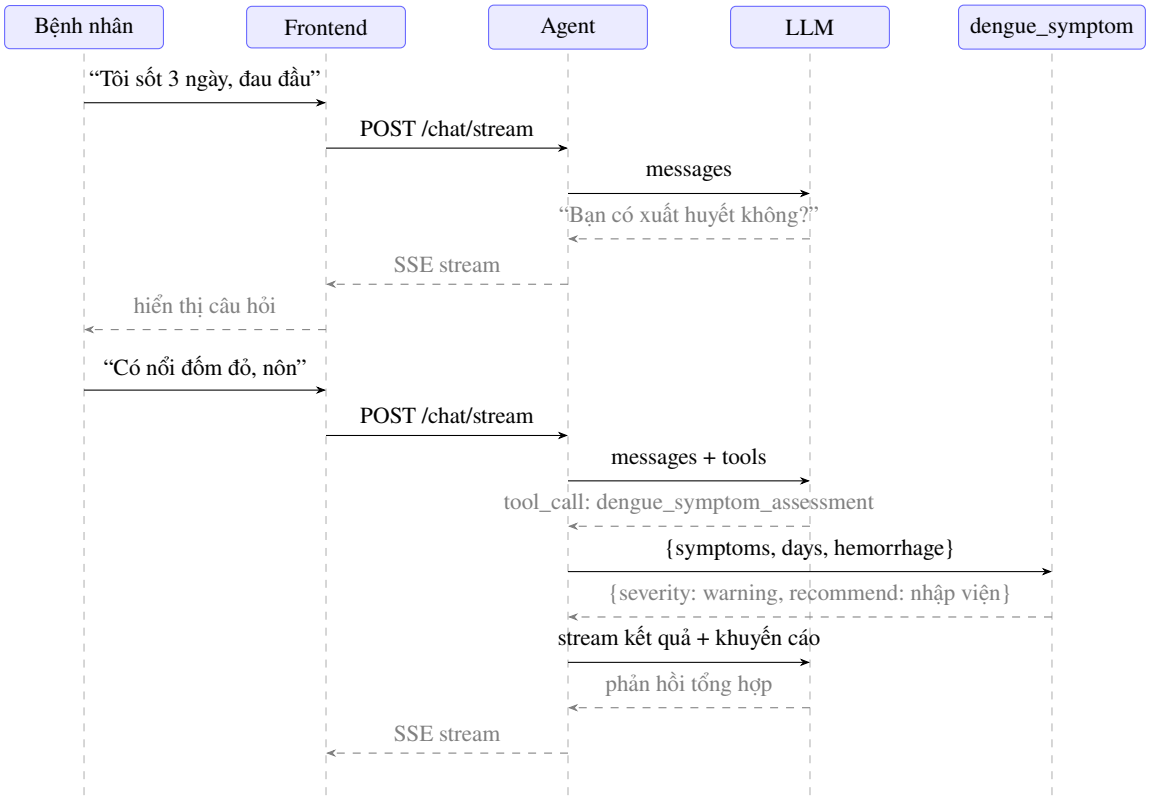
a, Sàng lọc theo triệu chứng (dengue_symptom_assessment)

Module triển khai bộ tiêu chí phân loại SXHD 4 lớp theo Phụ lục 2 của QĐ 2760/QĐ-BYT 2023:



Hình 2.6: Cây quyết định phân loại mức độ SXHD theo QĐ 2760

Hình 2.7 mô tả luồng hội thoại sàng lọc SXHD (Flow D) từ góc độ kỹ thuật:



Hình 2.7: Sơ đồ tuần tự Flow D — Sàng lọc SXHD qua hội thoại 2 lượt

b, Sàng lọc theo CBC (dengue_cbc_predict)

Module phân tích CBC áp dụng 3 tiêu chí cận lâm sàng chính theo QĐ 2760/QĐ-BYT 2023 trang 5:

1. **PLT (Tiểu cầu)**: < 100 K/μL là tiêu chí cận lâm sàng SXHD.
2. **WBC (Bạch cầu)**: Giảm dưới 4 K/μL (leukopenia).
3. **HCT (Hematocrit)**: Tăng $> 20\%$ so với baseline (cô đặc máu).

Kết quả đầu ra phân mức: unlikely / possible / likely / very_likely. **Chức năng này chỉ khả dụng với bác sĩ**, không bao giờ xuất hiện trong bộ công cụ của bệnh nhân.

2.4.2 Thiết kế module tra cứu số liệu dịch bệnh

a, Đặt vấn đề

Khi người dùng hỏi về tình hình dịch bệnh thực tế ("hiện có bao nhiêu ca sốt xuất huyết tại TP.HCM?"), bản thân LLM không thể trả lời chính xác: kiến thức của mô hình bị giới hạn tại thời điểm huấn luyện (knowledge cutoff) và mô hình có xu hướng *bịa số* (hallucination) khi thiếu dữ liệu. Module này cung cấp cho Agent một công cụ truy xuất số liệu dịch tễ **thật, theo thời gian thực** từ nguồn chính thống là Trung tâm Kiểm soát Bệnh tật TP.HCM (HCDC).

b, Đặc điểm nguồn dữ liệu HCDC

HCDC công bố bản tin dịch bệnh **định kỳ mỗi tuần** dưới dạng trang HTML. Khảo sát cho thấy hai đặc điểm quyết định khả năng bóc tách tự động:

1. **Định dạng URL đồng nhất**: mọi bản tin theo khuôn ...tinh-den-tuan-{tuần}{năm}-{mã}.html, trong đó phần mã (shortcode) là ngẫu nhiên nên không thể suy đoán.
2. **Định dạng nội dung đồng nhất**: các câu số liệu trong bài theo mẫu cố định ("ghi nhận N trường hợp...", "tích lũy ... là M ca"), cho phép trích xuất bằng biểu thức chính quy (regex).

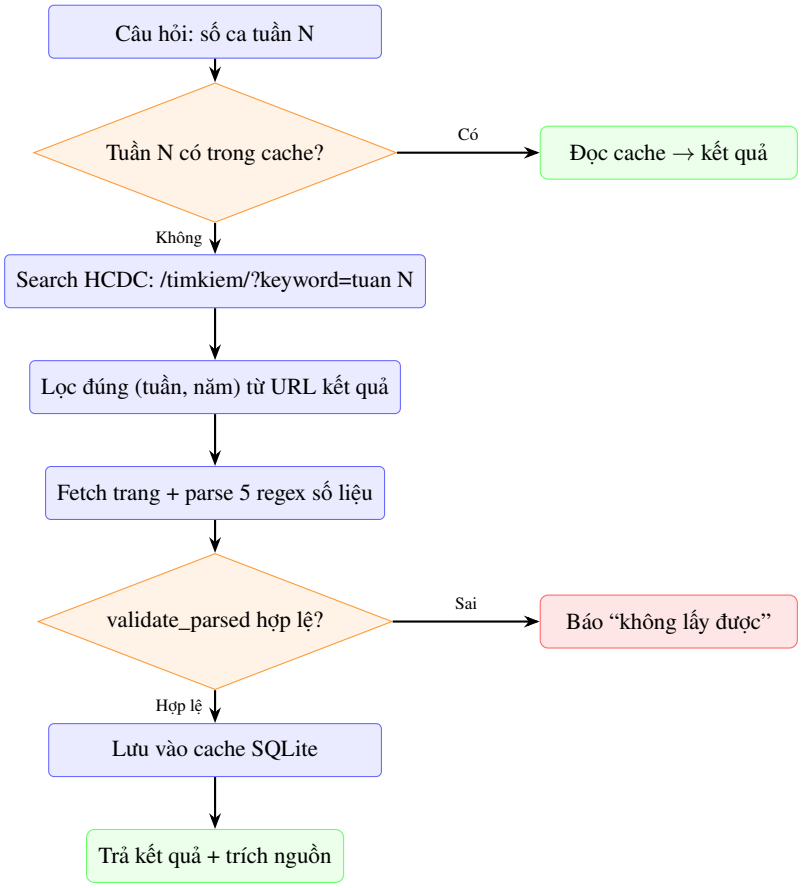
Hạn chế của nguồn: trang danh mục (category) của HCDC **liệt kê thiếu** một số tuần, không có sitemap cũng như API. Tuy nhiên ô tìm kiếm /timkiem/ cho phép truy vấn theo từ khóa "tuan N" và trả về liên kết bản tin tương ứng — đây là cơ chế then chốt để tra cứu *bất kỳ* tuần nào.

c, Kiến trúc module

Module được tổ chức thành subpackage tools/outbreak/ độc lập (cùng quy ước với tools/dengue/), gồm: công cụ public outbreak_get_stats, bộ scraper _hcdc_scraper và lớp cache lịch sử _cache (SQLite). Ba quyết định thiết kế chính:

- **Lazy-refresh thay vì lập lịch nền (scheduler)**: dữ liệu chỉ được tải khi người dùng hỏi và cache đã quá hạn (24 giờ), thay vì chạy tiến trình nền định kỳ. Cách này loại bỏ phụ thuộc thư viện lập lịch, phù hợp với tần suất công bố theo tuần của HCDC.
- **Cache lịch sử nhiều tuần**: mỗi tuần đã tra được lưu thành một dòng trong bảng SQLite (khóa chính disease + year + week). Cache vừa tăng tốc (không tải lại tuần đã có), vừa cho phép so sánh tuần-trên-tuần.
- **Tra cứu động (dynamic resolution)**: hỏi tuần nào thì tra tuần đó qua ô tìm kiếm HCDC, không cần khai báo (seed) danh sách URL thủ công.

Hình 2.8 mô tả luồng xử lý một truy vấn số liệu.



Hình 2.8: Luồng xử lý truy vấn số liệu dịch bệnh (lazy-refresh + tra cứu động)

d, Nguyên tắc chống bịa số

Để bảo đảm tính chính xác của số liệu y tế, module tuân thủ nguyên tắc **LLM không thực hiện tính toán trên số liệu**:

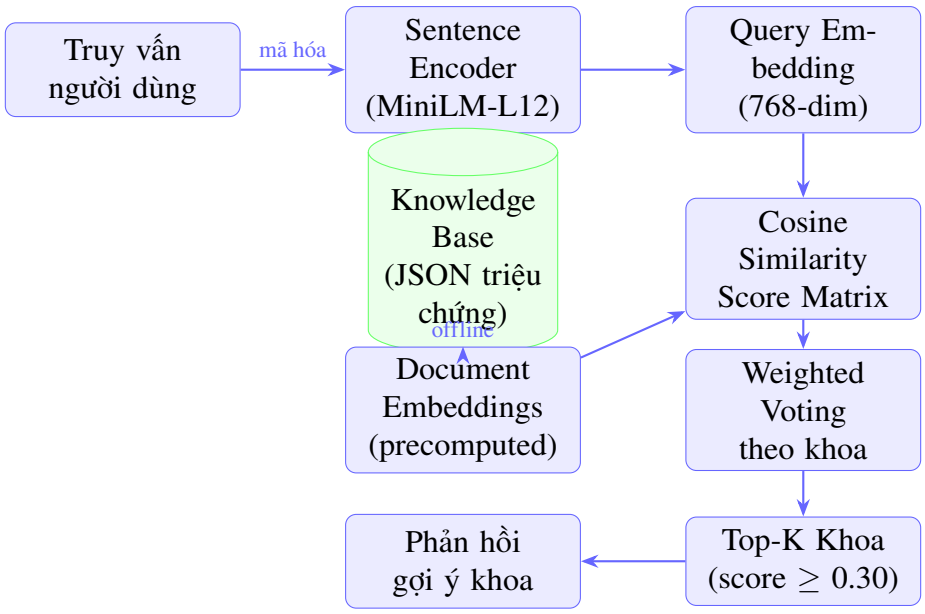
- Phép so sánh tăng/giảm giữa hai tuần được tính bằng mã Python và trả về dưới dạng chuỗi hoàn chỉnh (wow_text); LLM chỉ hiển thị lại nguyên văn, không tự cộng trừ.

- Hàm validate_parsed kiểm tra tính hợp lý của số liệu bóc tách (số ca trong khoảng hợp lệ, lũy kế $\geq$ số ca tuần); nếu sai, hệ thống báo "không lấy được" thay vì trả số sai.
- System prompt cấm LLM nội suy/ước lượng số ca của tuần không có dữ liệu, đồng thời bắt buộc trích dẫn nguồn (source_url) và ghi chú độ trễ, phạm vi dữ liệu (chỉ TP.HCM, theo tuần).

2.4.3 Thiết kế RAG cho gợi ý khoa

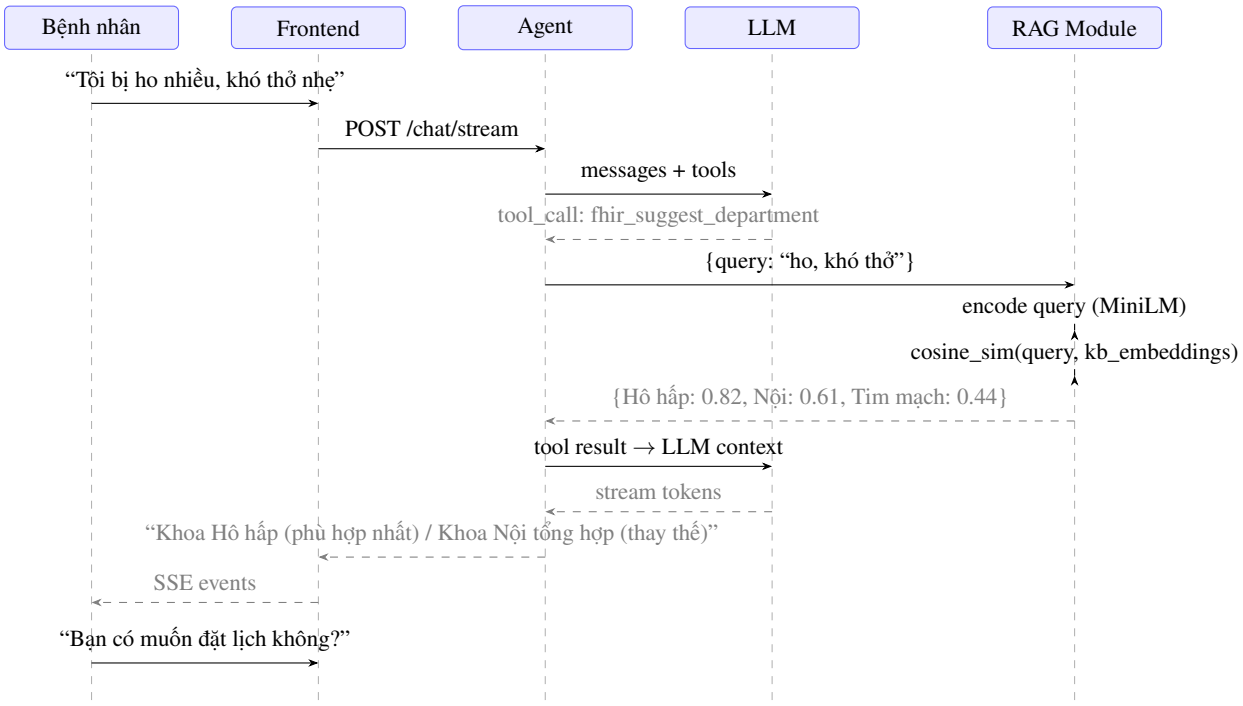
Hệ thống sử dụng kỹ thuật RAG [12] với mô hình paraphrase-multilingual-MiniLM-L12-v2 [11] để gợi ý khoa dựa trên triệu chứng:

1. Knowledge base: symptoms_departments.json mô tả triệu chứng và mã khoa.
2. Tính cosine similarity giữa query embedding và embedding các document triệu chứng.
3. Weighted voting: tổng điểm cosine theo khoa để xếp hạng.
4. Ngưỡng ≥ 0.30 để lọc kết quả không liên quan.



Hình 2.9: Pipeline RAG gợi ý khoa phù hợp dựa trên triệu chứng

Hình 2.10 mô tả Flow C — luồng gợi ý khoa thuần RAG khi bệnh nhân chỉ mô tả triệu chứng mà không yêu cầu đặt lịch ngay:



Hình 2.10: Sơ đồ tuần tự Flow C — Gợi ý khoa bằng RAG từ mô tả triệu chứng

2.4.4 Thiết kế module tra cứu thông tin thuốc (RAG truy hồi qua API)

Ngoài các luồng nghiệp vụ trên FHIR, hệ thống bổ sung một công cụ **tra cứu kiến thức thuốc** (drug_info) trả lời các câu hỏi dạng “liều dùng, tác dụng phụ, tương tác của thuốc X”. Đây là nhóm câu hỏi mà các trợ lý y khoa thương mại (ví dụ MedDeep AI) phục vụ bằng cách truy hồi tài liệu PubMed và trả lời kèm trích dẫn. Module này lấp khoảng trống đó cho HIS Agent, đồng thời giữ nguyên triết lý **dẫn nguồn để chống bịa** (hallucination) đã áp dụng cho module sàng lọc SXHD.

Hai biến thể RAG trong hệ thống. Hệ thống sử dụng hai kiểu Retrieval-Augmented Generation cho hai bài toán khác nhau (Bảng 2.3). Việc tách bạch này thể hiện nguyên tắc *chọn cơ chế truy hồi phù hợp với bài toán* thay vì áp dụng một kỹ thuật cho mọi trường hợp.

Tiêu chí	**RAG vector (dense)**	**RAG truy hồi qua API (search)**
Áp dụng cho	Gợi ý khoa	Tra cứu thông tin thuốc
Cơ chế truy hồi	Embedding + cosine similarity	Tìm kiếm từ khóa (lexical) qua API
Nguồn tri thức	Corpus nội bộ (symptoms_departments.json)	NCBI: StatPearls + PubMed
Hiểu ngữ nghĩa	Có (gần nghĩa)	Không (khớp từ ngữ)
Cập nhật nguồn	Tĩnh, dựng lại thủ công	Trực tuyến theo thời gian thực
Module	_rag_departments.py	drug_info

Bảng 2.3: So sánh hai biến thể RAG trong HIS Agent

Pipeline của drug_info gồm bốn bước:

1. **Truy hồi (Retrieval):** gọi NCBI E-utilities — esearch trên db=books (StatPearls) và db=pubmed — để tìm tài liệu liên quan đến tên thuốc.
2. **Lấy nội dung (Fetch):** efetch lấy abstract từng bài; esummary lấy tiêu đề và liên kết để dựng danh sách nguồn.
3. **Tăng cường (Augment):** ghép các đoạn bằng chứng (evidence_en) làm ngữ cảnh, mỗi đoạn được **gắn nhãn số nguồn** [n] khớp với danh sách nguồn.
4. **Sinh (Generation):** LLM tổng hợp câu trả lời tiếng Việt, **chỉ dựa trên bằng chứng** đã truy hồi, chèn số trích dẫn [n] sau mỗi ý.

Ràng buộc grounding. Để câu trả lời đáng tin và kiểm chứng được, thiết kế áp ba ràng buộc: (1) LLM chỉ được nêu thông tin có trong evidence_en, không bổ sung liều/phác đồ từ kiến thức nền; (2) số trích dẫn [n] phải khớp đúng nhãn nguồn (chống lệch trích dẫn); (3) không mô tả nguồn quốc tế là "hướng dẫn Việt Nam". Do tìm kiếm là *lexical* (khớp từ ngữ, không hiểu nghĩa), module bổ sung một bảng **từ đồng nghĩa** (ví dụ *paracetamol* → *acetaminophen*) để mở rộng truy vấn. Phạm vi tra cứu được giới hạn quanh danh mục 52 thuốc của hệ thống.

Giới hạn. Nguồn NCBI là tiếng Anh và không phải hướng dẫn kê đơn của Bộ Y tế Việt Nam; câu trả lời mang tính tham khảo, kèm khuyến cáo không thay thế chỉ định lâm sàng. Hướng phát triển là bổ sung nguồn trong nước (Dược thư Quốc gia Việt Nam) theo cùng kiến trúc dẫn nguồn.

2.4.5 Thiết kế module Sinh hiệu với Clinical Decision Support

Module sinh hiệu (vital signs) cho phép bác sĩ ghi nhận đầy đủ các chỉ số sống: huyết áp tâm thu/tâm trương, mạch, nhịp thở, nhiệt độ, SpO2, cân nặng, chiều cao. Mỗi lần đo

được lưu thành một FHIR Observation với category = vital-signs, sử dụng panel code LOINC 85353-1 (Vital signs panel), trong đó từng chỉ số con là một component[] với LOINC riêng (8480-6, 8462-4, 8867-4, v.v.).

Hệ thống cảnh báo 5 mức theo HL7 v3 ObservationInterpretation. Sau khi nhập, hệ thống tự động phân tích và đánh dấu mức bất thường:

- N (Normal) — trong khoảng bình thường, không cảnh báo.
- H/L (High/Low) — bất thường, cảnh báo màu vàng.
- HH/LL (Critical High/Low) — nguy hiểm, cảnh báo màu đỏ.

Ngưỡng cảnh báo dựa trên các nguồn quốc tế và Việt Nam đã được cite trong code (trường _citation):

- **Huyết áp**: QĐ 3192/QĐ-BYT 2010 [20] và Khuyến cáo VNHA 2022 (đồng bộ ESC/ESH 2018). Hệ thống lấy mức nặng hơn giữa tâm thu và tâm trương để xác định phân độ tăng huyết áp (THA độ 1: 140–159/90–99, THA độ 2: 160–179/100–109, THA độ 3 / cấp cứu: $\geq$180/$\geq$110).
- **Mạch, nhịp thở, SpO2**: National Early Warning Score 2 (NEWS2) do Royal College of Physicians UK ban hành năm 2017 [21]. Đây là chuẩn early warning được NHS UK sử dụng chính thức, có validation khoa học rộng rãi.
- **Nhiệt độ**: Harrison's Principles of Internal Medicine, 21st ed (2022) [22], chương "Approach to the patient with fever".

Ngoài cảnh báo, mỗi chỉ số trong Observation.component đều có interpretation (HL7 code) và referenceRange.text (khoảng tham chiếu kèm nguồn cite) để đảm bảo *interoperability* — mọi HIS khác đọc record này đều thấy được ngữ cảnh đánh giá.

2.4.6 Thiết kế Drug Catalog và Allergy Management

Drug Catalog. Hệ thống xây dựng catalog 52 thuốc thiết yếu Việt Nam dựa trên *Danh mục thuốc thiết yếu Bộ Y tế (QĐ 1165/QĐ-BYT 2022)* [23], phân loại theo 10 nhóm dược lý: Kháng sinh (Penicillin, Cephalosporin, Macrolide, Quinolone, Tetracycline, Sulfonamide, Lincosamide), Giảm đau–hạ sốt, NSAID, Tim mạch (CCB, ARB, ACE-I, Beta-blocker, Thiazide, Loop diuretic), Đái tháo đường, Tiêu hóa (PPI), Hô hấp, Vitamin, Rối loạn lipid (Statin), Chống đông (VKA). Mỗi entry có trường drug_class làm khóa cho cross-reactivity check, category để group trong dropdown UI, và max_daily_dose_mg cho cảnh báo quá liều.

AllergyIntolerance Resource (FHIR R5). Dị ứng của bệnh nhân được lưu dưới dạng AllergyIntolerance với đầy đủ field chuẩn HL7:

- clinicalStatus (active/inactive/resolved): trạng thái hiện tại của dị ứng.
- verificationStatus (confirmed/unconfirmed): đã xác nhận lâm sàng hay BN tự khai.
- category (medication/food/environment): phân nhóm chất gây dị ứng.
- criticality (low/high): tự sinh từ severity — "severe" → "high", còn lại "low".
- code.text: tên chất gây dị ứng (free text).
- reaction[].severity (mild/moderate/severe) và manifestation: chi tiết phản ứng.

Module này tuân thủ khung pharmacovigilance của WHO và Quyết định 1088/QĐ-BYT 2013 [24] về *Hướng dẫn theo dõi phản ứng có hại của thuốc*, đảm bảo audit trail cho ADR reporting.

2.4.7 Thiết kế module Kê đơn thuốc với 3-tier Allergy Check

Module kê đơn thuốc là điểm nhấn Clinical Decision Support, áp dụng mô hình *four-layer drug allergy check* của Hsieh TC et al. (AJHP 2004) [16] và pattern *three-tier alert system* của Epic SmartChart / Cerner PowerChart (Bates DW et al., JAMIA 2001) [17].

Cấu trúc FHIR MedicationRequest R5. Mỗi thuốc trong đơn là một MedicationRequest resource. Đơn thuốc có nhiều thuốc thì N record cùng groupIdentifier.value (UUID) để group lại. Sử dụng cú pháp R5 mới:

- medication.concept (CodeableReference) thay vì medicationCodeableConcept của R4.
- reason (CodeableReference, link được Condition) thay vì reasonCode.
- dosageInstruction.timing.repeat với frequency, periodUnit, boundsDuration cho cách dùng có cấu trúc.

Cơ chế check dị ứng 4 lớp. Trước khi cho phép kê 1 thuốc, hệ thống chạy thứ tự:

1. **Layer 1 — Direct name match**: "BN dị ứng Amoxicillin" + kê Amoxicillin → CRITICAL.
2. **Layer 2 — Same drug class**: "BN dị ứng Penicillin" + kê Amoxicillin (class=penicillin) → CRITICAL.
3. **Layer 3 — Pharmacologic cross-reactivity**: "BN dị ứng Penicillin" + kê Cefuroxime (class=cephalosporin) → WARNING với tỉ lệ phản ứng chéo 2–4.8% theo evidence post-1980 (StatPearls NBK459320 [19]).
4. **Layer 4 — Clear**: không phát hiện liên hệ, cho phép kê.

Bảy cặp cross-reactivity được hard-code trong tools/shared/_drug_allergy_rules.py, mỗi cặp cite nguồn open-access đã được verify (chi tiết quote nguyên văn trong thesis/drug_allergy_cross_reactivity_sources.md):

Cặp dị ứng – thuốc kê	**Mức**	**Nguồn verified**
Penicillin ↔ Cephalosporin	warning	StatPearls NBK459320 (NIH Bookshelf, open access)
NSAID ↔ Aspirin (AERD)	critical	Laidlaw TM 2024, PMC11695142
NSAID ↔ COX-2 selective	warning	Stevenson DD 2003, Stockley's [18]
Sulfa kháng sinh ↔ Thiazide	warning	Giles A et al 2019, PMC6789825
Sulfa kháng sinh ↔ Loop diuretic	warning	Giles A et al 2019
ACE-Inhibitor → ARB (angioedema)	warning	Sica DA & Black HR 2002, PMC8099384

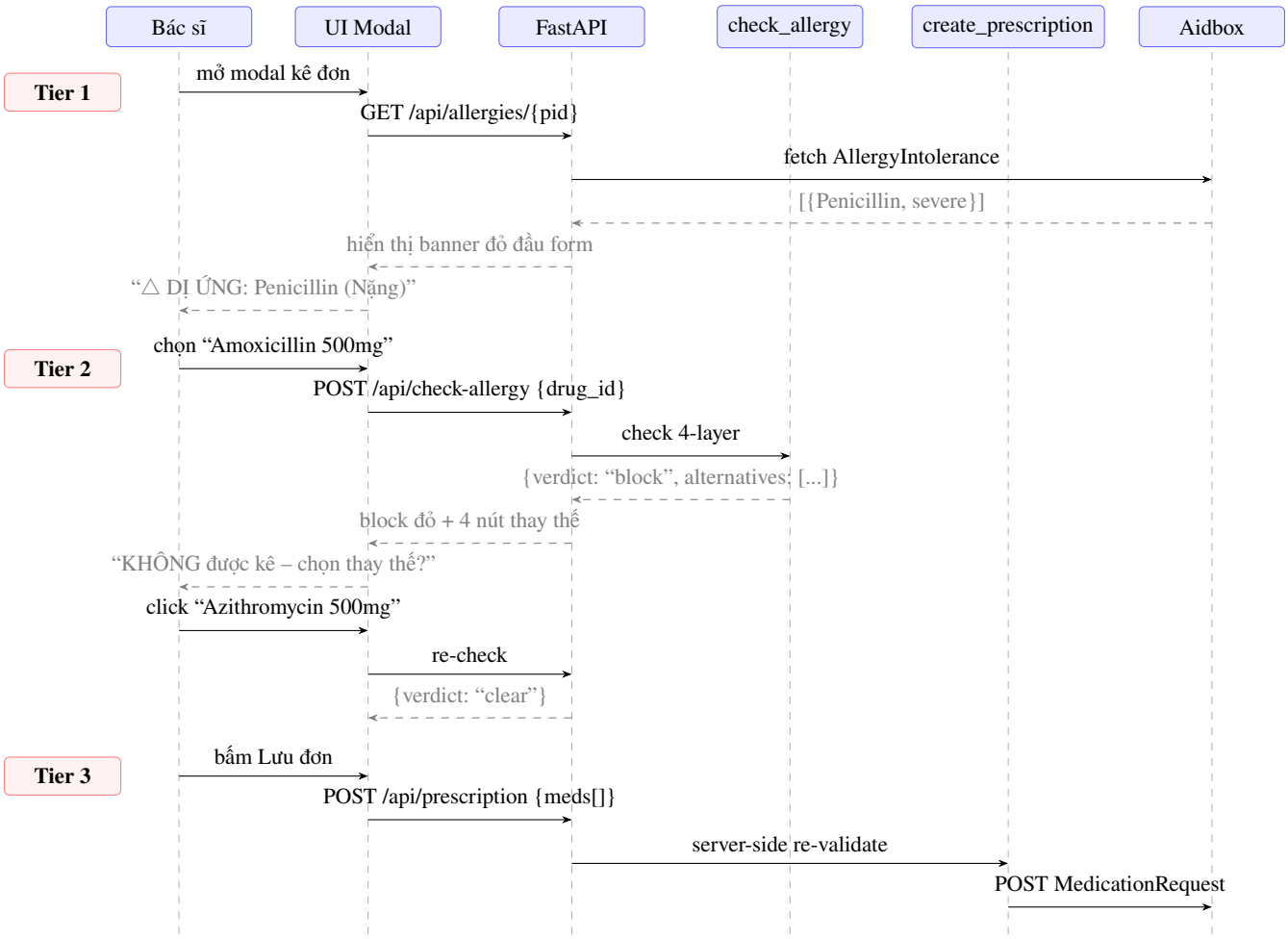
Bảng 2.4: Bảng 7 cặp cross-reactivity với nguồn open-access đã verify

3-Tier Alert System. Áp dụng pattern UI safety của HIS chuẩn quốc tế:

- **Tier 1 — Passive banner**: khi mở modal kê đơn, banner đỏ liệt kê toàn bộ dị ứng thuốc của BN ngay đầu form — phòng ngừa trước khi bác sĩ chọn thuốc.
- **Tier 2 — Active alert real-time**: khi bác sĩ chọn thuốc từ dropdown, hệ thống gọi fhir_check_drug_allergy 200ms, hiển thị block đỏ/vàng ngay dưới dropdown kèm đề xuất 4 thuốc thay thế cùng category nhưng khác drug_class. Bác sĩ có thể click nút thay thế để switch ngay.
- **Tier 3 — Hard stop**: khi submit đơn thuốc, server-side validate. Nếu có thuốc verdict "block" mà không có override_reason ($\geq$5 ký tự) thì chặn submit. Lý do override được lưu vào MedicationRequest.note để audit trail.

Đề xuất thuốc thay thế tuân theo *IDSA Clinical Practice Guideline 2012* cho Group A Streptococcal Pharyngitis (Shulman ST et al., CID 2012;55(10):e86-e102): khi BN dị ứng Penicillin, ưu tiên Macrolide (Azithromycin, Clarithromycin) hoặc Lincosamide (Clindamycin), tránh hoàn toàn beta-lactam.

Hình 2.11 mô tả sơ đồ tuần tự của 3-tier alert khi bác sĩ kê đơn cho bệnh nhân có dị ứng Penicillin:



Hình 2.11: Sơ đồ tuần tự 3-tier allergy check khi kê đơn cho BN dị ứng Penicillin

2.4.8 Thiết kế module Complete Visit Modal

Vấn đề thiết kế. Workflow "kết thúc khám" ban đầu rời rạc thành 2 bước: bác sĩ phải nhập chẩn đoán ở 1 modal (tạo ClinicalImpression), sau đó mở panel BN riêng để kê đơn (tạo MedicationRequest). Bác sĩ thường phải nhập *lại* chẩn đoán, dễ quên kê đơn, và 2 hành động không atomic.

Giải pháp. Tích hợp thành 1 modal đa năng complete-overlay với các thành phần:

- Banner Tier 1 dị ứng (đầu form) — tự load qua fhir_get_allergies khi mở modal.
- Nút ◇ **AI gợi ý** — gọi LLM với context (CBC, sinh hiệu, dị ứng đã có) để auto-fill 3 ô chẩn đoán + ghi chú + plan theo phác đồ BYT.
- Ô chẩn đoán + ghi chú lâm sàng (textarea).
- Section đơn thuốc (tùy chọn) với allergy check 3-tier đầy đủ — reuse component từ modal kê đơn riêng.
- Kế hoạch / tái khám.

- Checkbox ☐ Gửi email tóm tắt + field email auto-fill từ Patient.telecom[email] của FHIR.

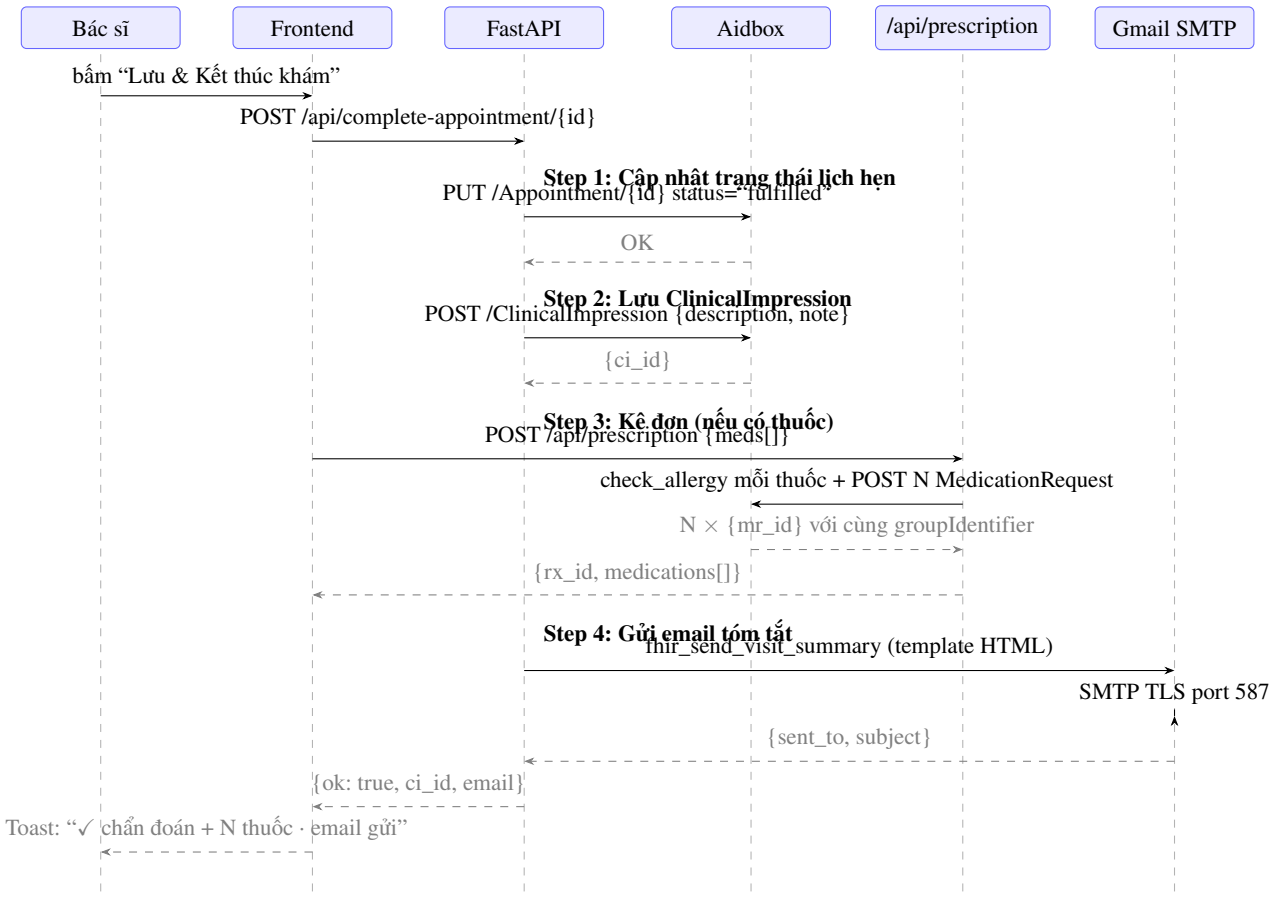
Khi bác sĩ bấm **Lưu & Kết thúc khám**, hệ thống thực hiện 4 hành động atomic trong 1 transaction:

1. PUT /Appointment/{id} đổi status = "fulfilled".
2. POST /ClinicalImpression với description = chẩn đoán, note[] chứa ghi chú và plan.
3. Nếu có đơn thuốc, POST /MedicationRequest cho từng thuốc với chung groupIdentifier và reason = chẩn đoán.
4. Gọi fhir_send_visit_summary để render template HTML và gửi qua Gmail SMTP TLS (port 587).

Template email tóm tắt khám. HTML có header màu thương hiệu BV, các section: chẩn đoán, ghi chú lâm sàng (white-space pre-line), kế hoạch tái khám, table đơn thuốc với cách dùng từng thuốc, footer ký tên BS. Đảm bảo BN nhận được kết quả khám chuyên nghiệp, có thể in/lưu để mang đến nhà thuốc.

Bệnh nhân xem kết quả qua UI. Trong panel "Hồ sơ của tôi", tab *Ghi chú bác sĩ* đã được refactor để gộp ClinicalImpression và MedicationRequest cùng ngày/cùng diagnosis vào 1 card duy nhất, giúp BN xem "tóm tắt lần khám" tự nhiên — chẩn đoán + đơn thuốc cùng chỗ thay vì rải ở 2 tab khác nhau.

Hình 2.12 mô tả luồng atomic 4 hành động khi bác sĩ bấm "Lưu & Kết thúc khám":



Hình 2.12: Sơ đồ tuần tự Complete Visit Modal — 4 hành động atomic khi bác sĩ kết thúc khám

2.5 Thiết kế xác thực và bảo mật người dùng

Hệ thống áp dụng chiến lược "lazy registration": lần đầu đăng nhập bằng số điện thoại, hệ thống tra cứu trong Aidbox (Practitioner hoặc Patient), tự động tạo tài khoản với mật khẩu mặc định. Mật khẩu được hash bằng bcrypt. SQLite lưu thông tin xác thực, tách biệt với dữ liệu lâm sàng trong FHIR.

Xác thực danh tính bệnh nhân qua Session Token. Sau khi đăng nhập thành công, server tạo một session token ngẫu nhiên (32 byte, secrets.token_urlsafe) và lưu ánh xạ token → {fhir_id, name, role} trong bộ nhớ RAM (api/tokens.py). Token được trả về cho frontend và lưu trong biến JavaScript _sessionToken. Mỗi tin nhắn tiếp theo, frontend đính kèm token trong HTTP header X-Session-Token.

Phía backend, trước khi gọi agent, hàm _resolve_patient() tra cứu token để lấy fhir_id thật của bệnh nhân, thay vì tin vào trường patient_id do frontend tự khai. fhir_id xác thực này sau đó được inject vào system prompt của agent:

```
if patient_id:
    system += "Benh nhan: " + patient_name
    system += " -- FHIR ID: " + patient_id
    system += " -- KHONG truy cap ho so BN khac."
```

Đoạn mã: Inject danh tính bệnh nhân vào system prompt.

Hai lớp bảo vệ patient portal. Để đảm bảo bệnh nhân chỉ truy cập được hồ sơ của chính mình, hệ thống áp dụng *defense-in-depth* với 2 lớp độc lập:

Lớp 1 — Prompt-level (LLM behavior). Hệ thống **prepend** một khối quy tắc "ƯU TIÊN TUYỆT ĐỐI" vào *đầu* system prompt mỗi request, override mọi flow mô tả phía dưới:

```
# (1) Tieu de cap uu tien cao nhat -- LLM uu tien hon
#     cac flow chi tiet dat phia duoi cua prompt
=== UU TIEN TUYET DOI ===

# (2) Inject ten + FHIR ID cua BN dang dang nhap.
#     Lay tu session token, KHONG tin frontend.
Benh nhan dang chat: Pham Alex (FHIR ID: c0f6...).

# (3) Cac quy tac CAM (negative rules) - LLM tuan theo
#     tot hon negative rule hon la positive rule dai

- TUYET DOI KHONG hoi 'Ten day du cua ban'.
  # Tranh viec LLM bi cac flow HOI ten o duoi keo theo

- TUYET DOI KHONG goi fhir_search_patient.
  # Khong can search vi da co patient_id san

- Khi dat lich: chi HOI EMAIL roi goi tool ngay.
  # Email la thong tin DUY NHAT can hoi them

- TUYET DOI KHONG tra cuu ho so BN khac.
  # Quy tac bao mat - lop bao ve so 1 (prompt-level)
```

Đoạn mã: Khối ưu tiên tuyệt đối được prepend ở đầu prompt khi BN đã đăng nhập.

Khối này được đặt ở đầu thay vì cuối prompt vì mô hình LLM thường tuân theo các quy tắc xuất hiện sớm trong context, đặc biệt khi các quy tắc đó được đánh dấu cấp ưu tiên cao ("ƯU TIÊN TUYỆT ĐỐI"). Thực nghiệm cho thấy việc append cuối prompt khiến LLM vẫn bị các flow chi tiết hơn (vd "HỎI tên → fhir_search_patient") kéo theo.

Lớp 2 — Tool-level enforcement (hard block). Đối với 6 tool đọc dữ liệu cá nhân, code chặn ở backend trước khi gọi tool:

```
# Danh sach 6 tool nhay cam can scope theo BN dang dang nhap.
# Bat ky tool nao trong set nay GOI BOI patient role
# se bi force patient_id ve BN dang login.
_PATIENT_SCOPED_TOOLS = {
    "fhir_get_patient_history",        # tien su benh
    "fhir_get_observations",           # sinh hieu, xet nghiem
    "fhir_get_patient_appointments",   # lich hen
    "fhir_cancel_appointment",         # huy lich
    "fhir_get_allergies",              # NEW: di ung
    "fhir_get_prescriptions",          # NEW: don thuoc
}

# Kiem tra truoc khi goi tool:
if role == "patient" and tool_name in _PATIENT_SCOPED_TOOLS:
    # Khach vang lai (chua login) - chan ngay
    if not patient_id:
        return {"error": "Can dang nhap"}

    # CRITICAL: override patient_id ve gia tri tu session token.
    # Du LLM co truyen ID khac (do prompt injection),
    # dong nay van ghi de ve dung BN dang login.
    args["patient_id"] = patient_id  # HARD FORCE
```

Đoạn mã: Force patient_id về BN đang đăng nhập (lớp bảo vệ thứ 2).

Lớp 2 không phụ thuộc vào LLM: kể cả nếu LLM bị prompt injection và cố gắng gọi fhir_get_observations(patient_id="id-cua-BN-khac"), code vẫn override về patient_id của BN đang đăng nhập. Đây là biện pháp *fail-safe* cho trường hợp lớp 1 thất bại.

Role-based Tool Registry. Ngoài 2 lớp trên, hệ thống còn áp dụng nguyên tắc *least privilege* qua dictionary _TOOLS_BY_ROLE: mỗi role chỉ "nhìn thấy" subset tool phù hợp khi LLM nhận danh sách tool. Patient role có 15 tool gồm 2 tool mới fhir_get_allergies và fhir_get_prescriptions để hỗ trợ truy vấn chính mình, nhưng **không có** fhir_search_patient — LLM không biết tool này tồn tại nên không thể gọi.

Với khách vãng lai (không có token hợp lệ), patient_id được đặt rỗng và agent sẽ hỏi tên khi cần tra cứu.

Ngoài ra, giao diện bệnh nhân bổ sung nút **Hồ sơ của tôi** trong thanh sidebar, chỉ hiển thị khi bệnh nhân đã đăng nhập (có currentPatient trong state). Khi nhấn, hệ thống gọi

openPatientPanel(currentPatient.id) để hiển thị slide-over panel chứa sinh hiệu, kết quả CBC và lịch hẹn của chính bệnh nhân đó.

CHƯƠNG 3. Xây dựng hệ thống

3.1 Môi trường và công nghệ sử dụng

Thành phần	Công nghệ	Ghi chú
LLM	Qwen3-VL-Plus	Qua Alibaba DashScope (OpenAI-compatible)
LLM SDK	OpenAI-compatible SDK	Tương thích DashScope, OpenRouter và Anthropic
Web framework	FastAPI 0.111+	ASGI, async, SSE
FHIR server	Aidbox (Medplum)	FHIR R5, Docker
Database	SQLite + bcrypt	Lưu tài khoản người dùng
RAG model	sentence-transformers	paraphrase-multilingual-MiniLM-L12-v2
HTTP client	httpx	Gọi Aidbox REST API
Frontend	React + TypeScript (Vite)	Giao diện SPA, cập nhật token theo SSE
Email	Gmail SMTP	Gửi xác nhận lịch hẹn
Python	3.11+	Type hints, asyncio

3.2 Cấu trúc thư mục dự án

```
his-agent/
|-- main.py              # Entry point (uvicorn)
|-- agent.py             # Tool-calling loop, chat_stream()
|-- system_prompt.py     # System prompts theo vai tro
|-- db.py                # SQLite user store (bcrypt)
|-- api/
|   |-- __init__.py      # FastAPI app factory + CORS
|   |-- sessions.py      # Session management (disk-
backed)
|   `-- routes/
|       |-- auth.py      # POST /api/login
|       |-- chat.py      # POST /chat/stream, session CRUD
|       `-- appointments.py
|-- tools/
|   |-- fhir/            # 26 cong cu FHIR (fhir_*.py)
|   |-- dengue/          # 2 cong cu SXHD
|   |-- outbreak/        # Tra cuu so lieu dich benh (HCDC)
|   |   |-- outbreak_get_stats.py  # Tool public (LLM goi)
|   |   |-- _hcdc_scraper.py       # Scrape + search HCDC
|   |   |-- _cache.py              # SQLite lich su nhieu tuan
|   |   `-- _mock_data.json        # Data demo offline
|   |-- shared/
|   |   |-- _aidbox_client.py
|   |   `-- _rag_departments.py
|   `-- knowledge_base/
`-- frontend/            # Giao dien React + TypeScript (Vite)
    |-- index.html
    |-- package.json     # React 18 + Vite + TypeScript
    `-- src/
        |-- App.tsx      # Routing + layout
        |-- api.ts       # Goi REST API + nhan SSE streaming
        |-- pages/       # 9 trang (Login, ChatPage, DoctorChat...)
        `-- components/  # 11 component (ChatPanel, CbcModal...)
```

Sơ đồ: Cấu trúc thư mục dự án.

3.3 Thiết kế cơ sở dữ liệu

3.3.1 Lớp dữ liệu lâm sàng — Aidbox FHIR R5

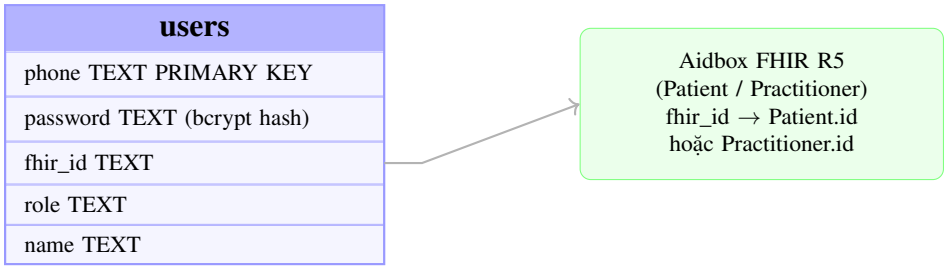
Dữ liệu lâm sàng được lưu trữ theo chuẩn FHIR R5 trong Aidbox. Các resource FHIR chính được sử dụng:

Resource FHIR	Mục đích	Trường chính
Patient	Hồ sơ bệnh nhân	name, telecom, birthDate, identifier (BHYT/CCCD)
Practitioner	Hồ sơ bác sĩ	name, telecom, qualification
Appointment	Lịch hẹn khám	start, end, status, participant, serviceType
Observation	Kết quả xét nghiệm	code (LOINC), value, effectiveDateTime, subject
DiagnosticReport	Phiếu xét nghiệm	result (Observation refs), conclusion, issued
Schedule	Lịch làm việc BS	actor, planningHorizon, serviceType
Slot	Khung giờ khám	schedule, start, end, status (free/busy)

Bảng 3.1: Các FHIR resource sử dụng trong hệ thống

3.3.2 Lớp xác thực — SQLite

Cơ sở dữ liệu SQLite lưu thông tin tài khoản người dùng, tách biệt hoàn toàn với dữ liệu lâm sàng:



Hình 3.1: Cấu trúc bảng SQLite và liên kết với FHIR resource

Chiến lược "lazy registration" hoạt động như sau: lần đầu đăng nhập với số điện thoại, hệ thống tra cứu trong Aidbox tìm Practitioner hoặc Patient có telecom khớp. Nếu tìm thấy, tự động tạo bản ghi trong SQLite với mật khẩu mặc định và fhir_id tương ứng. Người dùng có thể đổi mật khẩu ở lần đăng nhập tiếp theo.

3.4 Cài đặt lớp Agent

3.4.1 Nạp động công cụ

Hệ thống nạp động tất cả công cụ từ tools/fhir/ và tools/dengue/ khi khởi động:

```
def _load_tool_modules() -> dict[str, Any]:
    modules = {}

    # Thu muc goc chua tools (tools/fhir/, tools/dengue/)
    tools_dir = Path(__file__).parent / "tools"

    # Quet 2 subpackage voi pattern file tuong ung
    subpackages = {"fhir": "fhir_*.py", "dengue": "dengue_*.py"}

    for pkg, pattern in subpackages.items():
        # Iterate qua tat ca file Python match pattern
        for path in sorted((tools_dir / pkg).glob(pattern)):
            # Import dynamically vd: tools.fhir.fhir_create_vitals
            mod = importlib.import_module(f"tools.{pkg}.{path.stem}"

            # Chi nap module co convention TOOL_NAME + run()
            # -> tu dong skip helper file (_*.py)
            if hasattr(mod, "TOOL_NAME") and hasattr(mod, "run"):
                modules[mod.TOOL_NAME] = mod  # Dang ky vao registry

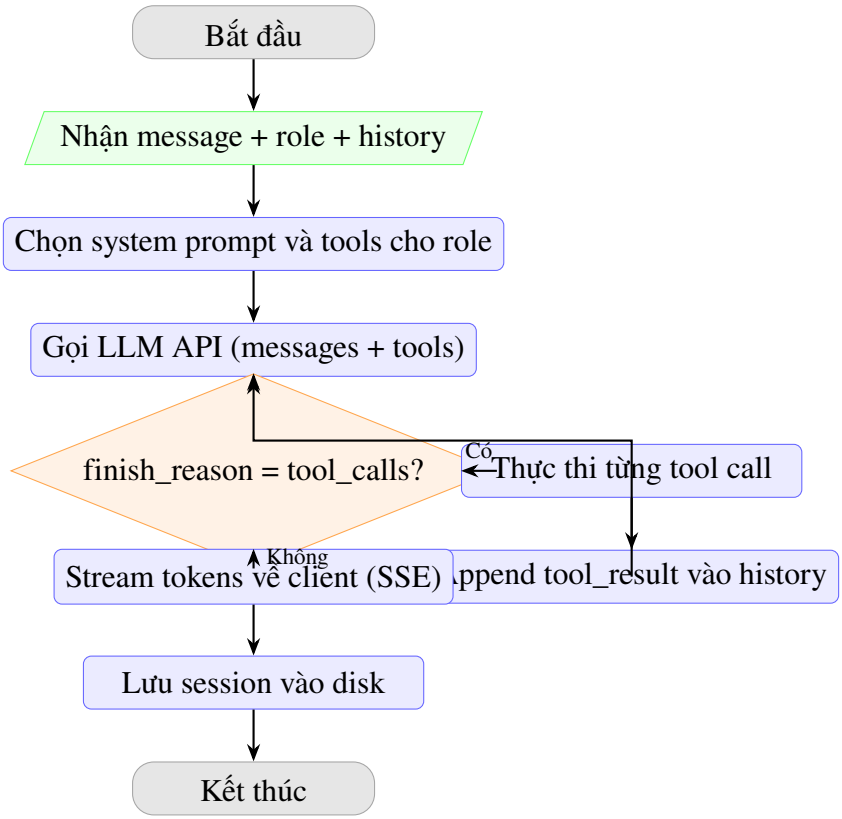
    return modules  # Dict {tool_name: module}
```

Đoạn mã: Nạp động công cụ từ filesystem.

Điều này cho phép thêm công cụ mới chỉ cần tạo file Python mới theo convention, không cần sửa core.

3.4.2 Sơ đồ luồng Agent Loop

Hình 3.2 mô tả chi tiết vòng lặp tool-calling của agent:



Hình 3.2: Lưu đồ vòng lặp Agent (Tool-Calling Loop)

3.4.3 Hỗ trợ ảnh (Vision) cho phân tích CBC

Khi bác sĩ upload ảnh phiếu xét nghiệm, nội dung được chuyển thành multipart content:

```
def _build_user_content(message, image_base64, image_mime):
    # Khong co anh -> tra ve text thuan (string)
    if not image_base64:
        return message

    # Co anh -> tra ve list theo chuan OpenAI multimodal:
    return [
        # Phan 1: text huong dan LLM phai lam gi voi anh
        {
            "type": "text",
            "text": message or "Day la ket qua xet nghiem CBC."
        },

        # Phan 2: anh dang base64 data URL
        # (LLM Qwen3-VL doc anh va trich xuat gia tri)
        {
            "type": "image_url",
            "image_url": {
                # Format: data:<mime>; base64,<encoded_image>
                "url": f"data:{image_mime}; base64,{image_base64}"
            }
        },
    ]
```

Đoạn mã: Xây dựng multipart content hỗ trợ vision.

LLM đọc ảnh, trích xuất chỉ số CBC (PLT, WBC, HCT, HGB, AST, ALT), rồi tự động gọi fhir_create_cbc để lưu FHIR và phân tích.

3.5 Cài đặt module sàng lọc SXHD

3.5.1 Phân tích CBC (dengue_cbc_predict)

```
# === Tieu cau (Platelets) - K/microL ===
# Tieu cau giam la dau hieu chinh cua SXHD pha nguy hiem
if plt < 20:    cbc_score += 1; warning_score += 1  # Cuc nang
elif plt < 50:  cbc_score += 1; warning_score += 1  # Nang
elif plt < 100: cbc_score += 1                       # Giam (chuan Q

# === Bach cau (WBC) - K/microL ===
# Leukopenia (bach cau giam) la tieu chi can lam sang SXHD
if wbc < 4.0: cbc_score += 1

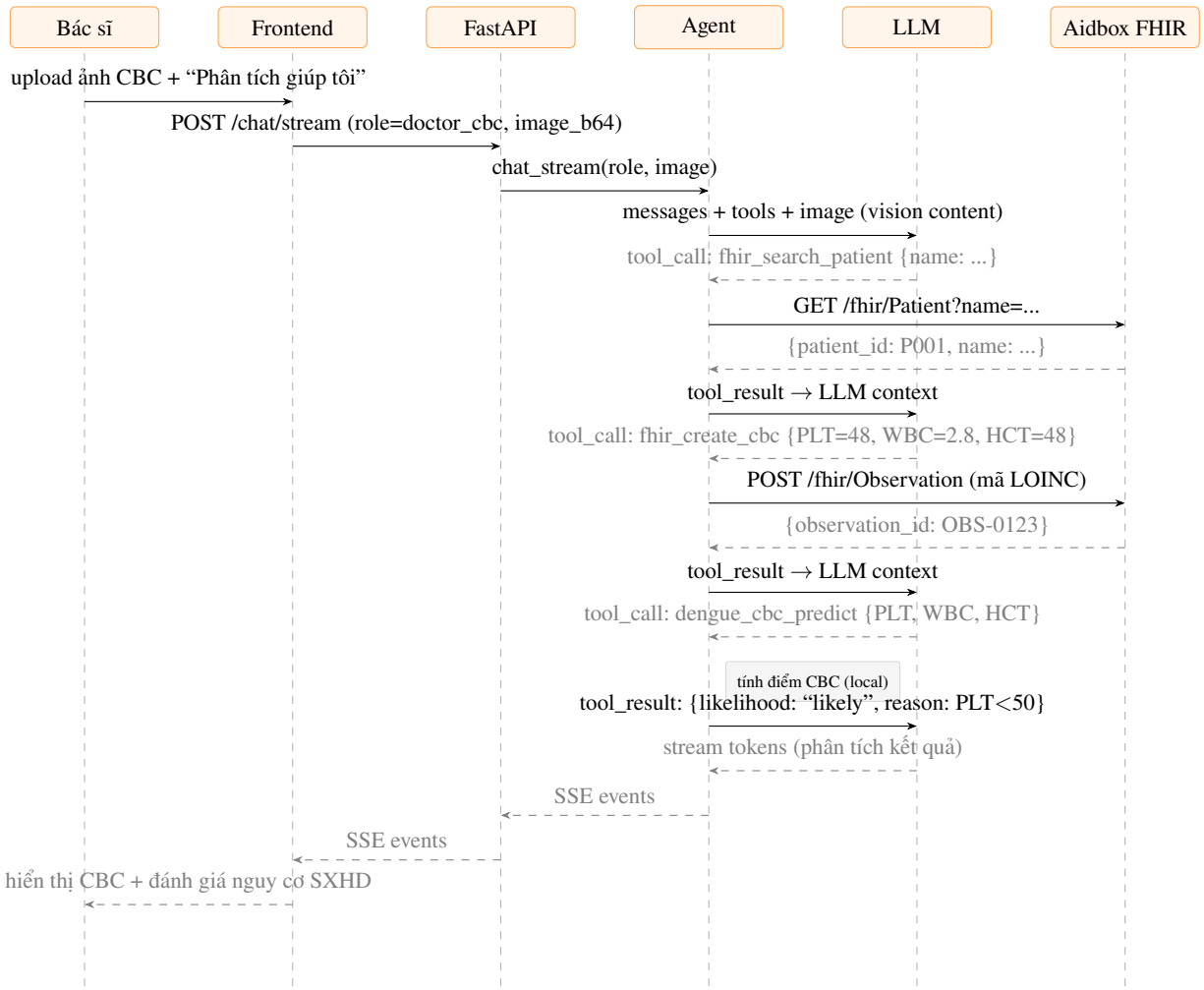
# === Hematocrit (HCT) - % ===
# Tang HCT > 20% so voi baseline => co dac mau (canh bao nang)
hct_change = (hct - baseline) / baseline * 100
if hct_change >= 20:   cbc_score += 1; warning_score += 2
elif hct_change >= 10: cbc_score += 1; warning_score += 1

# === Tong hop muc do nguy co ===
# severe_flag: co dau hieu nang lam sang truoc do (vd soc, xuat huye
if severe_flag or warning_score >= 2:
    likelihood = "very_likely"   # Nguy co rat cao -
> nhap vien gap
elif cbc_score >= 2:
    likelihood = "likely"        # Nguy co cao -
> theo doi sat
elif cbc_score >= 1:
    likelihood = "possible"      # Co dau hieu -
> xet nghiem them
else:
    likelihood = "unlikely"      # Khong co dau hieu
```

Đoạn mã: Logic đánh giá mức độ nguy cơ từ CBC.

3.5.2 Luồng phân tích CBC từ ảnh (Flow E — Bác sĩ)

Hình 3.3 mô tả luồng bác sĩ upload ảnh phiếu xét nghiệm CBC. Đây là luồng **dành riêng cho bác sĩ**: dengue_cbc_predict không có trong toolset bệnh nhân và nút upload ảnh bị ẩn trong giao diện bệnh nhân (role=patient).



Hình 3.3: Sơ đồ tuần tự Flow E — Bác sĩ phân tích CBC từ ảnh upload (Doctor Only)

3.6 Cài đặt module tra cứu số liệu dịch bệnh

Module được cài đặt trong subpackage tools/outbreak/ gồm 4 file, **không phát sinh thư viện phụ thuộc mới** (chỉ dùng httpx sẵn có và sqlite3 của thư viện chuẩn; không cần beautifulsoup4 hay thư viện lập lịch).

3.6.1 Bóc tách dữ liệu bằng regex hai tầng

Tầng 1 — nhận diện liên kết bản tin trong trang HTML bất kỳ (trang tìm kiếm hoặc trang danh mục). Biểu thức (\d{1,2})(\d{4}) tách phần "{tuần}{năm}" dính liền: nhờ neo 4 chữ số ở năm, tuần một chữ số (vd tuan-52026 = tuần 5/2026) vẫn tách đúng mà không nhầm với tuần hai chữ số.

```
_LINK_RE = re.compile(
    r'href="([^"]*tinh-den-tuan-(\d{1,2})(\d{4})-
[^"]*\.html)"')
```

Tầng 2 — trích số liệu từ nội dung bài (sau khi loại bỏ thẻ HTML, chuẩn hóa khoảng trắng) bằng 5 mẫu cố định ở Bảng 3.2.

Trường	Mẫu nhận dạng (rút gọn)
Tuần / năm	tính đến tuần (N)/(YYYY)
Số ca tuần	ghi nhận (số) trường hợp mắc bệnh sốt xuất huyết
Lũy kế	Tổng số ca ... tích lũy ... là (số) ca
Xu hướng	sốt xuất huyết (giảm|tăng|ổn định) (số)%
Ngày cập nhật	Cập nhật: HH:MM - DD/MM/YYYY

Bảng 3.2: Năm mẫu regex trích số liệu từ nội dung bản tin HCDC

Số liệu kiểu Việt Nam được chuẩn hóa khi parse: dấu "." là phân cách hàng nghìn (18.136 → 18136), dấu "," là thập phân (5,9% → 5.9).

3.6.2 Tra cứu động và xử lý nhiễu tìm kiếm

Để tra một tuần bất kỳ, module gửi truy vấn tới ô tìm kiếm HCDC với từ khóa "tuan N". Do tìm kiếm trả kết quả *mờ* (vd "tuan 5" lẫn cả tuần 50, 51, 52), bước lọc **bắt buộc khớp chính xác** cặp (tuần, năm) bóc từ URL — kết quả nhiễu bị loại bỏ:

```
def search_bulletin_url(week, year):
    resp = httpx.get(SEARCH_URL, params={"keyword": f"tuan {week}"},
    # _extract_bulletins -> {(year, week): url}; chi lay dung (year,
    return _extract_bulletins(resp.text).get((year, week))
```

Tuần mới nhất được lấy từ trang danh mục (chọn cặp (năm, tuần) lớn nhất); các tuần lịch sử lấy qua tìm kiếm. Cả hai dùng chung hàm _extract_bulletins.

3.6.3 Cache lịch sử và so sánh tuần-trên-tuần

Mỗi tuần tra được lưu vào bảng outbreak_history (SQLite, khóa chính disease+year+week, INSERT OR REPLACE). Phép so sánh với tuần liền trước dùng truy vấn SQL lấy tuần gần nhất nhỏ hơn, kết hợp kiểm tra đúng liền kề (week − 1), rồi tính chênh lệch bằng Python:

```
prev = get_previous(disease, year, week)   # ORDER BY year,week DESC
if prev and prev.week == week - 1:
    delta = cur_cases - prev_cases          # vd 409 -
397 = 12
    pct   = round(delta / prev_cases * 100, 1)
    wow_text = f"tang {delta} ca (+{pct}%) so voi tuan {prev.week}"
```

Chuỗi wow_text hoàn chỉnh được đưa thẳng vào kết quả tool; LLM chỉ render lại, bảo đảm con số không bị mô hình tính sai.

3.7 Cài đặt module tra cứu thông tin thuốc (drug_info)

Module được cài đặt trong subpackage tools/knowledge/ và được **nạp động** nhờ quy ước đặt tên: hàm nạp công cụ của Agent quét thêm mẫu *_info.py, nên chỉ cần thêm file mới mà không sửa lõi. Công cụ được cấp cho cả vai trò bệnh nhân và bác sĩ. Toàn bộ truy hồi dùng httpx sẵn có, không phát sinh thư viện phụ thuộc mới.

3.7.1 Truy hồi từ NCBI bằng tìm kiếm từ khóa

Module gọi NCBI E-utilities qua hai cơ sở dữ liệu: db=books (StatPearls — chuyên luận thuốc) và db=pubmed (bằng chứng nghiên cứu). Vì tìm kiếm là *lexical* (khớp từ ngữ, không hiểu nghĩa), truy vấn được mở rộng bằng bảng từ đồng nghĩa và bổ sung từ khóa theo khía cạnh hỏi (liều, tác dụng phụ, tương tác):

```
_SYNONYMS = {"paracetamol": "acetaminophen", "lasix": "furosemide",
syn  = _SYNONYMS.get(drug.lower())
base = f"({drug} OR {syn})" if syn else drug
term = f"{base} AND adverse effects"        # vd khia canh "tac dung
ids  = esearch(db="pubmed", term=term, retmax=3, sort="relevance")
```

Khóa API NCBI (NCBI_API_KEY) được nạp tùy chọn từ biến môi trường, nâng giới hạn gọi từ 3 lên 10 request/giây — cần thiết vì mỗi lần tra thực hiện nhiều lời gọi liên tiếp.

3.7.2 Gắn nhãn nguồn cho bằng chứng — chống lệch trích dẫn

Phiên bản đầu gộp toàn bộ abstract thành một khối ngữ cảnh duy nhất. Hệ quả: LLM không biết câu nào thuộc tài liệu nào nên **tự đánh số lại**, dẫn đến chip trích dẫn trỏ sai nguồn. Bảng 3.3 minh họa lỗi quan sát được với truy vấn "tác dụng phụ của cefuroxime".

Câu trả lời (trích)	Trước	Sau
"siêu nhiễm *Ps. aerugi-nosa*/enterococci"	[2] (sai)	**[3] (đúng)**
"viêm nội nhãn (endophthalmitis)"	—	[2] (đúng)
"chi tiết liều xem StatPearls"	—	[1] (đúng)

Bảng 3.3: Sửa lỗi lệch số trích dẫn nhờ gắn nhãn nguồn cho bằng chứng

Cách khắc phục: lấy abstract **theo từng bài** rồi **gắn nhãn** [n] (khớp số thứ tự nguồn) vào đầu mỗi đoạn bằng chứng. LLM được hướng dẫn dùng đúng nhãn này, không đánh số lại:

```
for pid in pmids:
    n += 1
    sources.append({"n": n, "title": title, "url": pubmed_url(pid)})
    abstract = efetch(db="pubmed", id=pid, rettype="abstract")  # tu
    evidence.append(f"[{n}] " + abstract)   # gan nhan -
> trich dan khong lech
```

Kết hợp với ràng buộc grounding trong system prompt (chỉ nêu thông tin có trong bằng chứng, không tự ghi liều), module trả lời trung thực: khi abstract không nêu liều, câu trả lời chủ động chỉ tới chuyên luận StatPearls thay vì bịa con số.

3.7.3 Hiển thị thẻ nguồn và chip trích dẫn

Khi một công cụ trả về trường sources, lớp Agent phát một sự kiện SSE sources để giao diện dựng **thẻ nguồn** (kèm liên kết) — cơ chế dùng chung cho mọi công cụ có dẫn nguồn:

```
result = json.loads(result_str)
if result.get("sources"):
    yield sse({"type": "sources", "items": result["sources"]})
```

Giao diện lưu danh sách nguồn, dựng hàng thẻ phía trên câu trả lời, và thay mỗi số [n] trong văn bản bằng một chip bấm được trỏ tới đúng nguồn:

```
html = html.replace(/\[(\d+)\]/g, (_m, n) => {
  const s = sources[parseInt(n) - 1];
  return `<a class="cite-chip" href="${s.url}" target="_blank">${n}<
});
```

Kết quả là giao diện trả lời có thẻ nguồn và chip trích dẫn bấm được, tương tự các trợ lý y khoa thương mại, nhưng mọi phát biểu đều truy ngược được về tài liệu gốc.

3.8 Cài đặt module Sinh hiệu, Dị ứng và Đơn thuốc

Giai đoạn 2 mở rộng hệ thống với 3 module lâm sàng cốt lõi mới và refactor workflow "kết thúc khám", tổng cộng **10 file Python mới** (1500 dòng code) và **6 endpoint REST mới**.

3.8.1 Cài đặt module Sinh hiệu (fhir_create_vitals)

Tool nhận đầu vào là dict các chỉ số sinh hiệu (huyết áp, mạch, nhiệt độ, SpO2, ...) và thực hiện 4 bước:

1. **Phân tích cảnh báo**: gọi 5 hàm _interp_temp, _interp_hr, _interp_rr, _interp_spo2, _interp_bp để xác định interpretation code (N/H/HH/L/LL) và message tiếng Việt cho mỗi chỉ số.

2. **Build FHIR Observation panel**: tạo resource với category = vital-signs, code LOINC 85353-1, mỗi chỉ số là 1 component[] có code (LOINC), valueQuantity (UCUM unit), interpretation (HL7 code), và referenceRange.text (cite nguồn).
3. **POST lên Aidbox** qua fhir_post.
4. **Trả về kết quả** gồm vitals_saved (dict tóm tắt), alerts[] (list cảnh báo theo severity "warning"/"critical"), và summary text.

Riêng huyết áp được xử lý đặc biệt: hàm _interp_bp nhận cả tâm thu và tâm trương, phân độ riêng từng giá trị, sau đó lấy mức nặng hơn để xác định alert chung (theo nguyên tắc của BYT 3192). VD sys=145, dia=80 → "THA độ 1 (145/80 mmHg)" do tâm thu đạt mức THA độ 1.

3.8.2 Cài đặt module Allergy Check 4 lớp (fhir_check_drug_allergy)

Tool nhận patient_id và drug_id (hoặc drug_name free text), trả về một dict gồm:

- verdict: "block" / "warn" / "clear".
- checks[]: list các vi phạm tìm thấy, mỗi entry có severity, type (direct_match / same_class / cross_reactivity / unmatched_allergy), allergy, message.
- suggested_alternatives[]: list tối đa 5 thuốc thay thế cùng category nhưng khác drug_class.

Thuật toán duyệt từng dị ứng đang hoạt động của bệnh nhân và kiểm tra tuần tự bốn lớp theo mức rủi ro giảm dần (trùng tên → cùng nhóm dược lý → phản ứng chéo → an toàn), sau đó lấy mức cảnh báo nặng nhất. Nguyên tắc thiết kế là *ưu tiên an toàn cho bệnh nhân hơn là bỏ sót cảnh báo*:

```
allergies = fhir_get_allergies(patient_id, active_only=True)
checks = []

for a in allergies:
    if a.category != "medication":      # chi xet di ung thuoc
        continue
    # Lop 1 -- Trung ten truc tiep -> critical
    if substance.lower() == drug.name.lower():
        checks.append(critical("direct_match", substance)); continue
    # Lop 2 & 3 -- Anh xa nhom duoc ly, kiem tra phan ung cheo
    for ac in match_allergy_classes(substance):
        rule = check_cross_reactivity(ac, drug.drug_class)
        if rule:            # cung nhom -
> critical; cheo -> warning
            checks.append(rule)

# Lop 4 -- Tong hop verdict: lay muc nang nhat
if   any(c.severity == "critical" for c in checks): verdict = "block
elif any(c.severity == "warning"  for c in checks): verdict = "warn"
else:                                               verdict = "clear
```

Đoạn mã: Pseudocode kiểm tra dị ứng 4 lớp (rút gọn).

Hàm match_allergy_classes tra cứu dictionary ALLERGY_ALIASES để map free text (vd "Bactrim", "Augmentin", "Sulfa") về drug_class chuẩn. Hàm check_cross_reactivity tra cứu dictionary CROSS_REACTIVITY chứa 7 cặp đã cite nguồn (bảng 2.4). Quy tắc same-class được tự sinh trong code (không hard-code vì có 50+ class).

3.8.3 Cài đặt module Kê đơn (fhir_create_prescription)

Tool nhận patient_id, practitioner_id, diagnosis, và list medications[] (mỗi item gồm drug_id, dose_per_intake, frequency_per_day, duration_days, instructions, override_reason). Quy trình:

1. Validate từng thuốc: lookup drug trong catalog, gọi fhir_check_drug_allergy. Nếu verdict="block" mà không có override_reason → **từ chối toàn bộ đơn** (return error: "prescription_blocked" với list blocked[]).
2. Sinh group_id = uuid.uuid4() làm khóa group cho cả đơn.

3. POST từng MedicationRequest với groupIdentifier.value = group_id chung. Cấu trúc R5: medication.concept (CodeableReference), reason (link diagnosis), dosageInstruction.timing.repeat có frequency, periodUnit="d", boundsDuration (UCUM "d"). Nếu có override_reason, thêm vào note[].text với prefix "△ Override cảnh báo dị ứng".
4. Trả về kết quả gồm prescription_id, list thuốc đã tạo, list warnings (nếu có), summary.

3.8.4 Cài đặt Complete Visit Modal và Email tóm tắt

Backend refactor. Endpoint POST /api/complete-appointment/{id} được mở rộng để nhận thêm 3 field: send_email (bool), patient_email (str), medications[] (list dict). Sau khi tạo ClinicalImpression, code:

1. Nếu send_email == true và có patient_email: fetch FHIR Patient (lấy tên) và Practitioner (lấy tên BS), sau đó gọi fhir_send_visit_summary với template HTML.
2. Frontend song song gọi POST /api/prescription nếu có thuốc.
3. Trả về JSON gộp cả clinical_impression_id và email status.

Email template sử dụng inline CSS để tương thích Gmail/Outlook, có:

- Header với màu thương hiệu BV.
- Section chẩn đoán (font đậm).
- Section ghi chú lâm sàng (white-space: pre-line để giữ format).
- Section kế hoạch tái khám.
- Bảng đơn thuốc với 2 cột "Thuốc" và "Cách dùng".
- Footer với chữ ký BS và disclaimer.

Frontend handler submitCompleteAppt() gồm: collect medications từ DOM, validate Tier 3 (chặn nếu có block chưa override), build emailMeds (snapshot drug_name + instruction để truyền vào template), gọi tuần tự /api/complete-appointment rồi /api/prescription. Toast hiển thị status email ở cuối.

3.9 Cài đặt API với FastAPI

3.9.1 Endpoint chat streaming

Endpoint dùng mẫu *producer–consumer* qua hàng đợi (asyncio.Queue) để chạy agent đồng bộ bên trong server bất đồng bộ mà không chặn event loop:

```
@router.post("/chat/stream")
async def chat_stream_endpoint(body: ChatRequest):
    session_id = _resolve_session(body)
    history = list(_load_session(session_id))
    loop = asyncio.get_event_loop()
    queue = asyncio.Queue()

    def producer():            # chay agent (sync) trong thread rien
        for chunk in chat_stream(body.message, history, ...):
            loop.call_soon_threadsafe(queue.put_nowait, chunk)
        loop.call_soon_threadsafe(queue.put_nowait, None)   # sentir

    async def generate():      # consumer: yield chunk ra client
        ThreadPoolExecutor(1).submit(producer)
        while (item := await queue.get()) is not None:
            yield item
        _save_session(session_id, history)

    return StreamingResponse(generate(), media_type="text/event-
stream")
```

Đoạn mã: Triển khai SSE streaming endpoint (rút gọn).

3.10 Giao diện người dùng

Hệ thống cung cấp hai giao diện web tách biệt, xây dựng bằng React + TypeScript (Vite) với SSE streaming để hiển thị phản hồi token-by-token.

3.10.1 Giao diện tổng quan

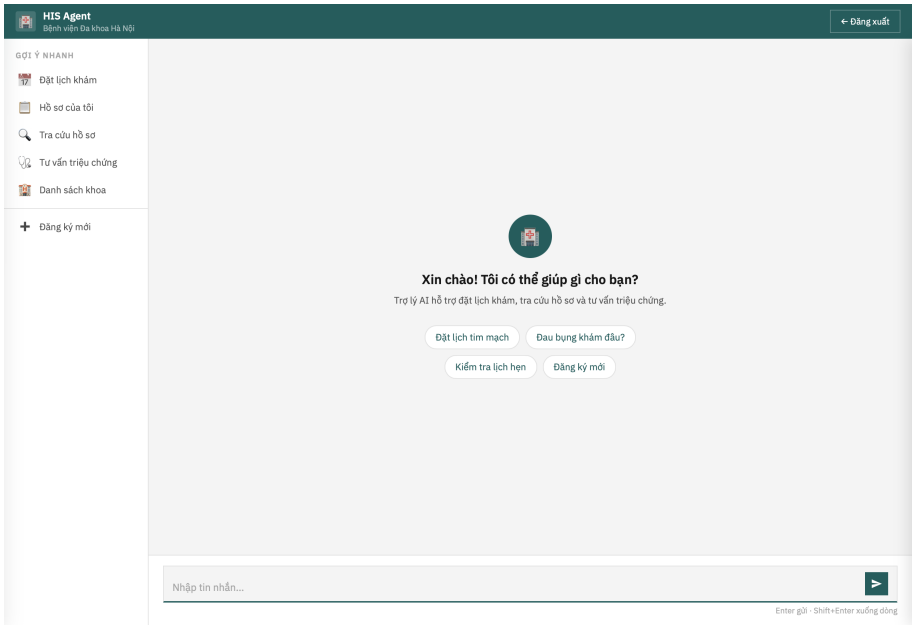
Hệ thống có hai giao diện tách biệt theo vai trò: giao diện bệnh nhân (Hình 3.4) tập trung vào hội thoại đơn giản, còn giao diện bác sĩ (Hình 3.5) bổ sung các công cụ lâm sàng và nút upload ảnh xét nghiệm.

3.10.2 Kết quả các chức năng phía bệnh nhân

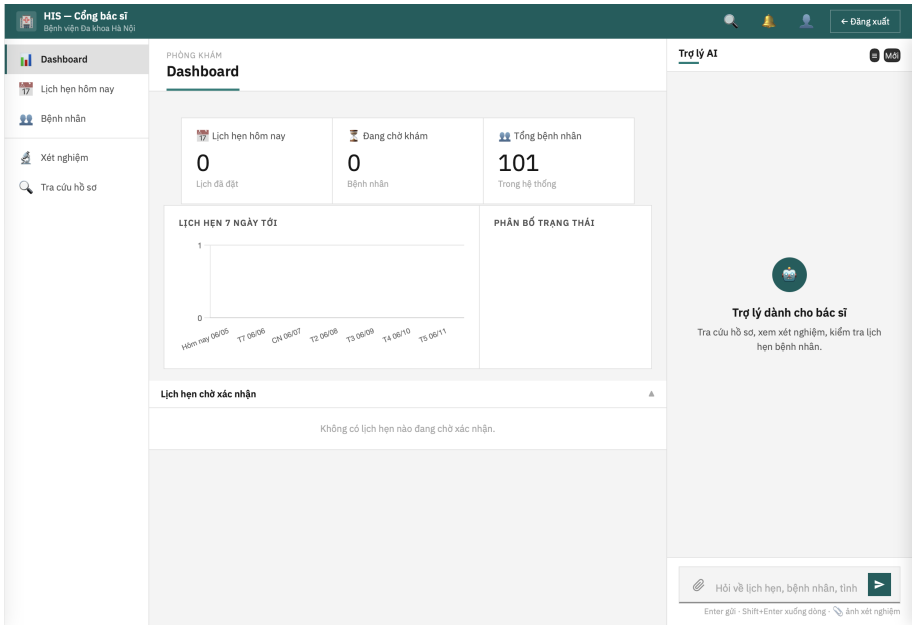
Các Hình 3.6–3.11 minh hoạ những chức năng tự phục vụ của bệnh nhân: từ phân loại triệu chứng, đặt lịch, sàng lọc dịch bệnh, tới tra cứu tri thức có dẫn nguồn.

3.10.3 Kết quả các chức năng hỗ trợ quyết định lâm sàng (bác sĩ)

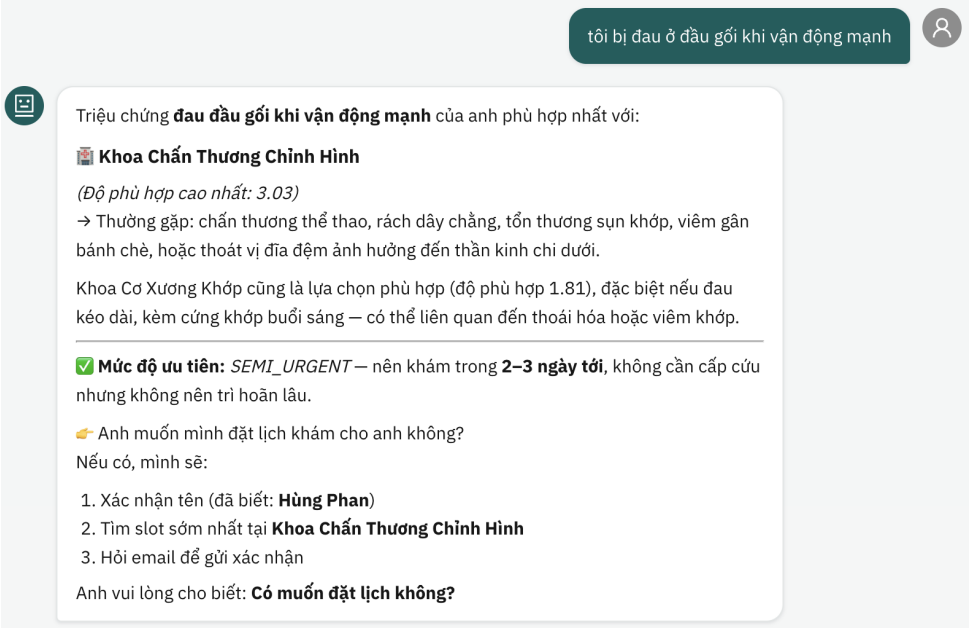
Các Hình 3.12–3.14 minh hoạ ba chức năng Clinical Decision Support nổi bật phía bác sĩ.



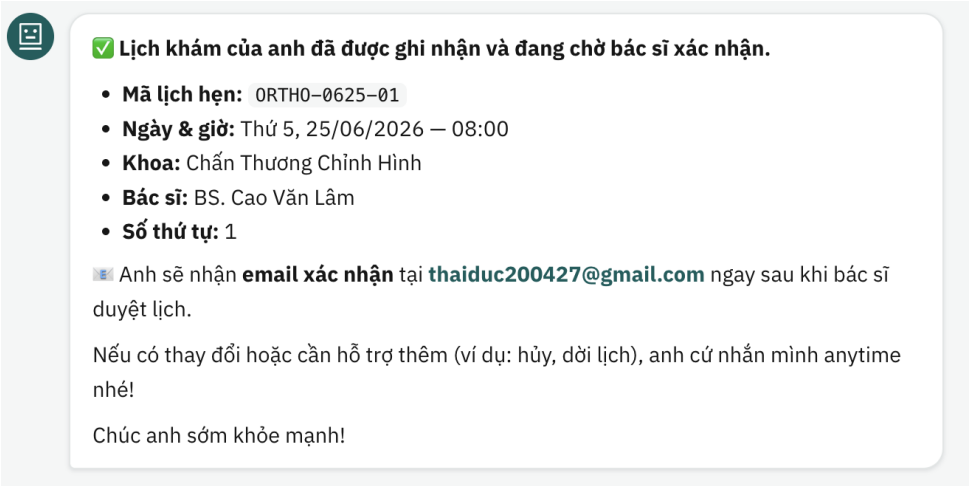
Hình 3.4: Giao diện hội thoại bệnh nhân



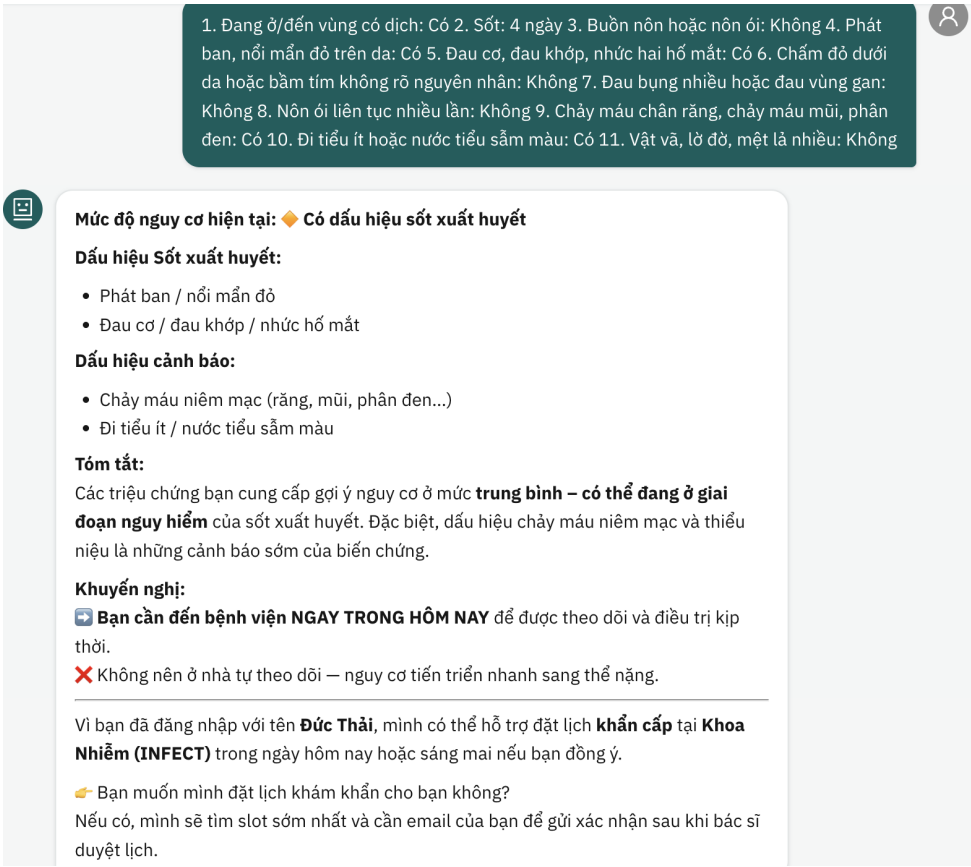
Hình 3.5: Giao diện tổng quan dành cho bác sĩ



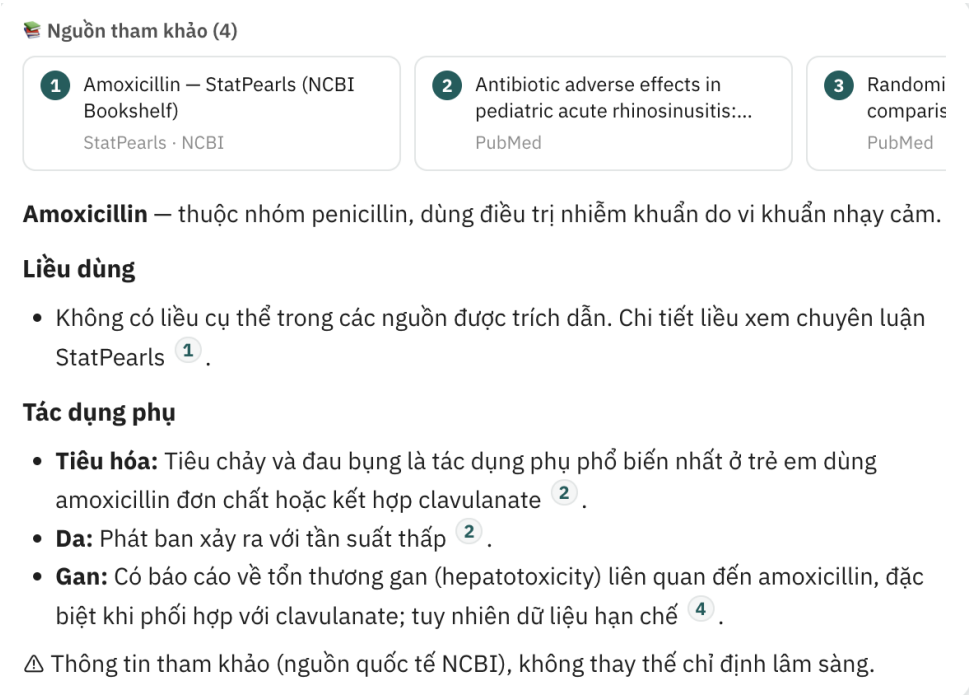
Hình 3.6: Phân loại mức độ ưu tiên (triage) và gợi ý khoa khám bằng RAG (F-P04, F-P05)



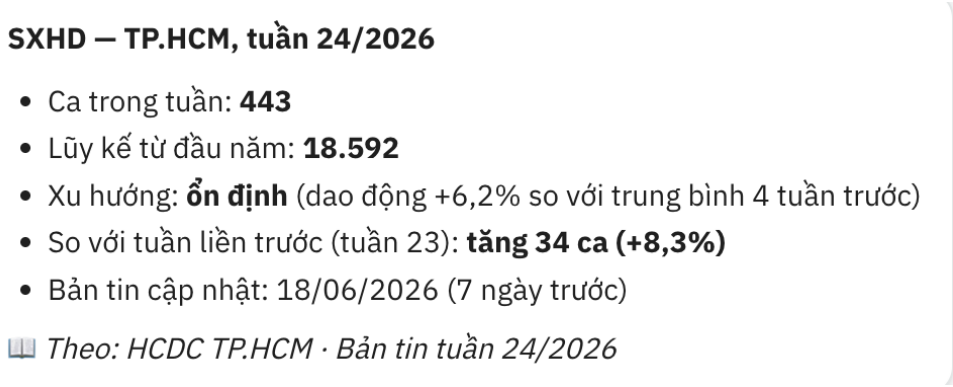
Hình 3.7: Đặt lịch khám ngay trong hội thoại (F-P01)



Hình 3.8: Sàng lọc sốt xuất huyết từ triệu chứng với phân độ cảnh báo theo QĐ 2760 (F-P06)



Hình 3.9: Tra cứu thông tin thuốc kèm thẻ nguồn NCBI và chip trích dẫn $[n]$ kiểm chứng được (F-P12)

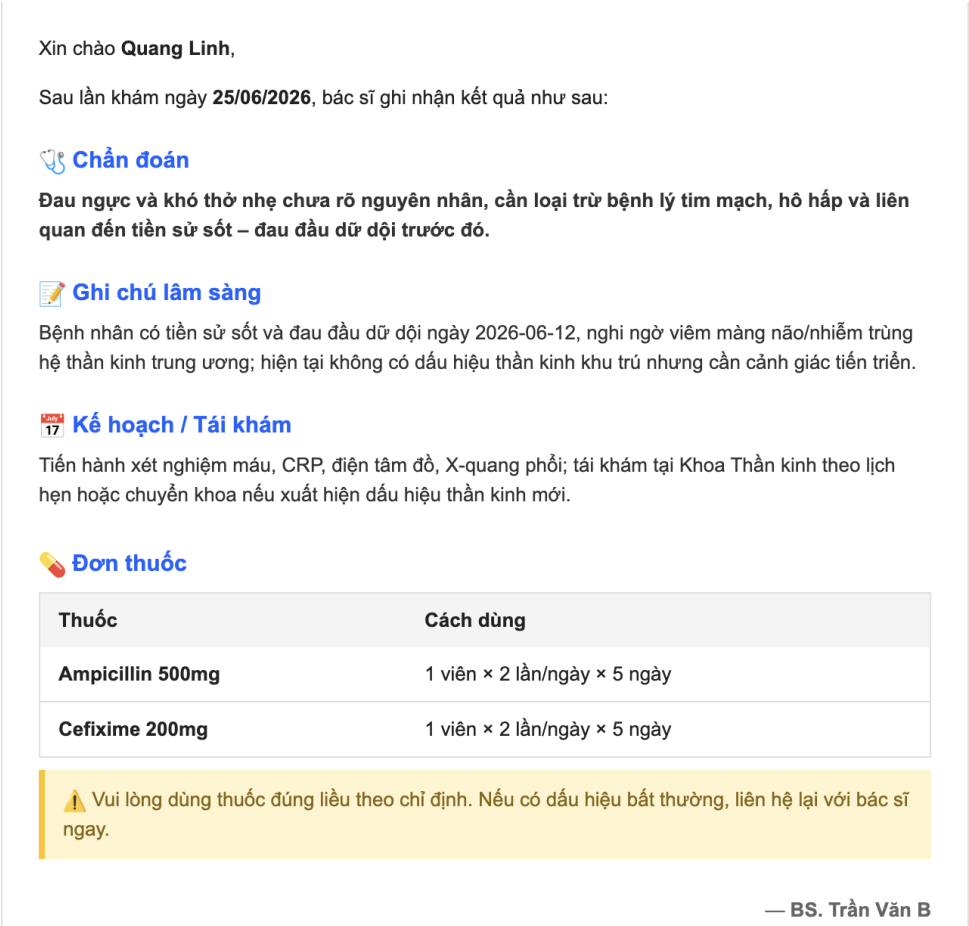


SXHD — TP.HCM, tuần 24/2026

- Ca trong tuần: **443**
- Lũy kế từ đầu năm: **18.592**
- Xu hướng: **ổn định** (dao động +6,2% so với trung bình 4 tuần trước)
- So với tuần liền trước (tuần 23): **tăng 34 ca (+8,3%)**
- Bản tin cập nhật: 18/06/2026 (7 ngày trước)

📖 *Theo: HCDC TP.HCM · Bản tin tuần 24/2026*

Hình 3.10: Tra cứu số ca sốt xuất huyết thực tế từ HCDC kèm trích dẫn nguồn (F-P13)



Xin chào **Quang Linh**,

Sau lần khám ngày **25/06/2026**, bác sĩ ghi nhận kết quả như sau:

🩺 **Chẩn đoán**

Đau ngực và khó thở nhẹ chưa rõ nguyên nhân, cần loại trừ bệnh lý tim mạch, hô hấp và liên quan đến tiền sử sốt – đau đầu dữ dội trước đó.

📝 **Ghi chú lâm sàng**

Bệnh nhân có tiền sử sốt và đau đầu dữ dội ngày 2026-06-12, nghi ngờ viêm màng não/nhiễm trùng hệ thần kinh trung ương; hiện tại không có dấu hiệu thần kinh khu trú nhưng cần cảnh giác tiến triển.

📅 **Kế hoạch / Tái khám**

Tiến hành xét nghiệm máu, CRP, điện tâm đồ, X-quang phổi; tái khám tại Khoa Thần kinh theo lịch hẹn hoặc chuyển khoa nếu xuất hiện dấu hiệu thần kinh mới.

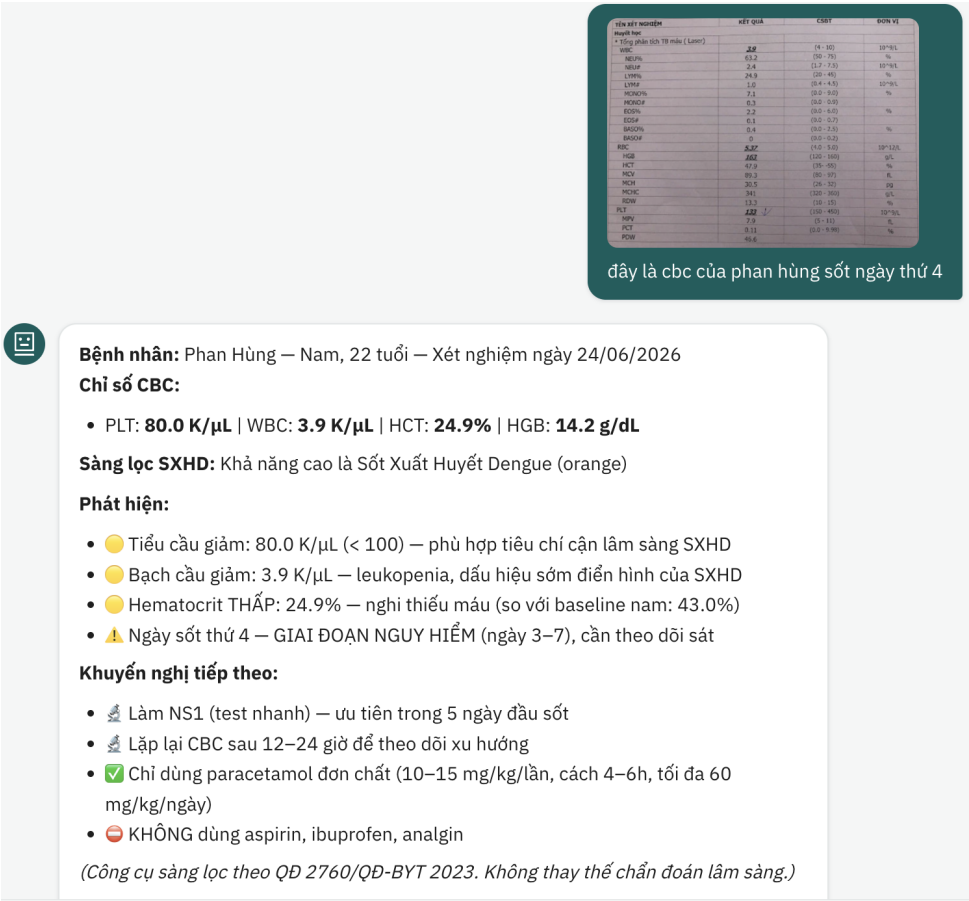
💊 **Đơn thuốc**

Thuốc	Cách dùng
Ampicillin 500mg	1 viên × 2 lần/ngày × 5 ngày
Cefixime 200mg	1 viên × 2 lần/ngày × 5 ngày

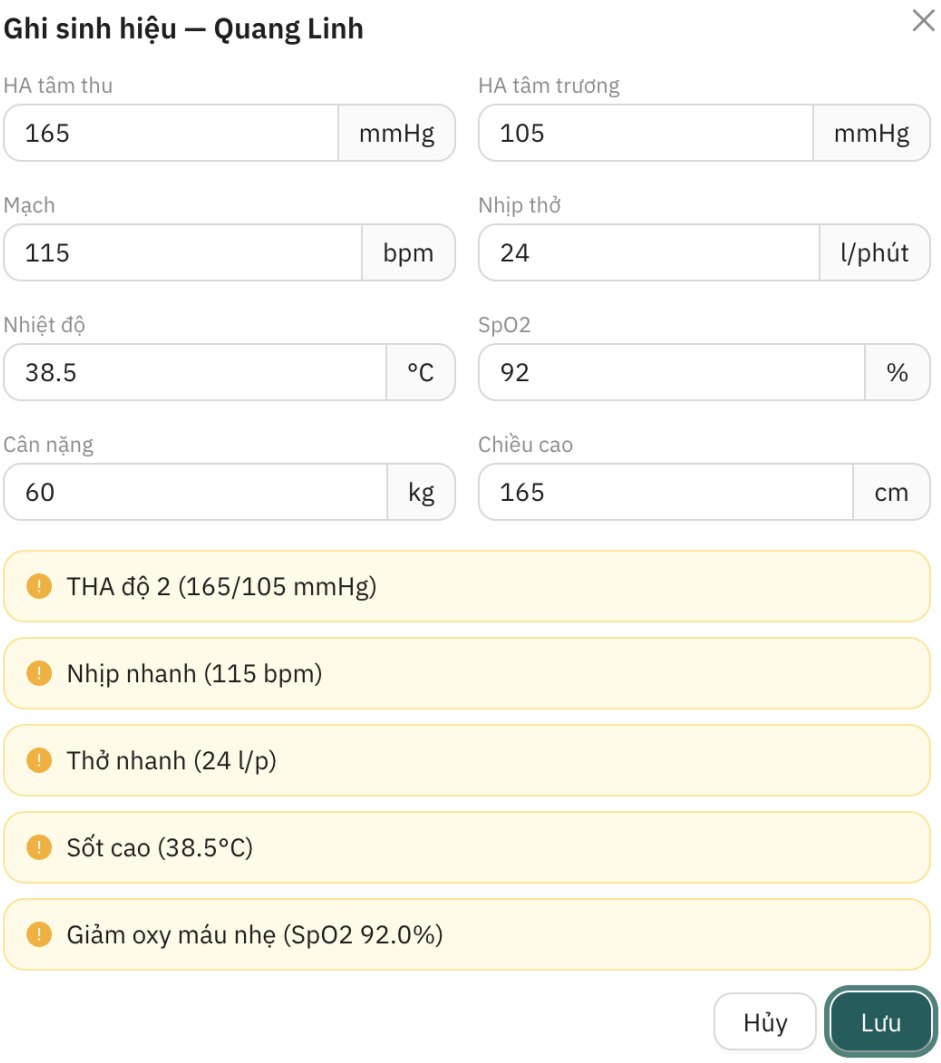
⚠️ Vui lòng dùng thuốc đúng liều theo chỉ định. Nếu có dấu hiệu bất thường, liên hệ lại với bác sĩ ngay.

— BS. Trần Văn B

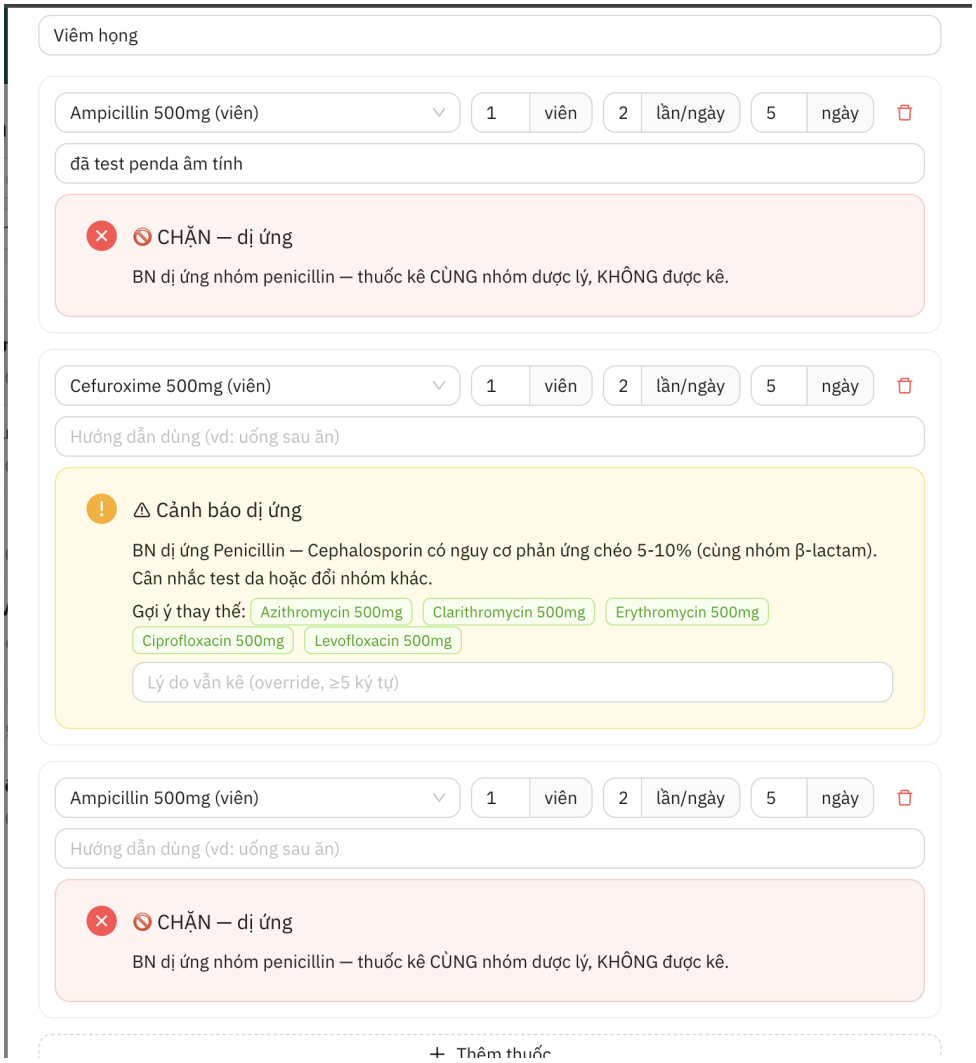
Hình 3.11: Email tóm tắt lần khám gửi tự động cho bệnh nhân (F-P11)



Hình 3.12: Phân tích chỉ số CBC và đánh giá nguy cơ sốt xuất huyết (F-D03)



Hình 3.13: Ghi nhận sinh hiệu với cảnh báo bất thường tự động theo NEWS2 và QĐ 3192 (F-D07)



Hình 3.14: Cảnh báo dị ứng khi kê đơn — phản ứng chéo Penicillin–Cephalosporin (F-D09)

CHƯƠNG 4. Kiểm thử, đánh giá và triển khai

Chương này trình bày quá trình kiểm thử các module chức năng, đánh giá độ chính xác và hiệu năng của hệ thống, cùng phương án triển khai thực tế.

4.1 Kiểm thử hệ thống

4.1.1 Đánh giá độ chính xác module sàng lọc SXHD

Module dengue_symptom_assessment được đánh giá theo hai tầng: (1) kiểm thử đơn vị trực tiếp trên logic phân loại; (2) kiểm thử end-to-end qua LLM với mô tả triệu chứng bằng tiếng Việt tự nhiên.

a, Tầng 1 — Kiểm thử đơn vị (Unit Test)

20 ca kiểm thử được xây dựng dựa trên tiêu chí phân độ SXHD của QĐ 2760/QĐ-BYT 2023, bao phủ đủ 4 mức độ và các trường hợp biên (sốt đúng 7 ngày, đúng 2 triệu chứng, dấu hiệu cảnh báo bypass cổng dịch tễ). Đầu vào là các tham số có cấu trúc truyền thẳng vào hàm run(), không qua LLM.

Mức độ	Số ca	Đúng	Sai	Accuracy
severe	4	4	0	100%
warning	5	5	0	100%
mild	5	5	0	100%
none	6	6	0	100%
Tổng	**20**	**20**	**0**	**100%**

Bảng 4.1: Kết quả kiểm thử đơn vị module sàng lọc SXHD (20 ca)

Kết quả 100% xác nhận rằng cây quyết định phân loại được cài đặt đúng theo QĐ 2760/QĐ-BYT 2023.

b, Tầng 2 — Kiểm thử end-to-end qua LLM

15 ca kiểm thử sử dụng mô tả triệu chứng bằng tiếng Việt tự nhiên (như bệnh nhân thực tế nhập vào chatbot). LLM phân tích ngữ nghĩa, trích xuất tham số và gọi tool dengue_symptom_assessment. Đây là tầng đánh giá khả năng hiểu ngôn ngữ tự nhiên của hệ thống agent.

Mức độ	Số ca	Đúng	Precision	Recall	F1
severe	3	3	100%	100%	1.00
warning	4	4	100%	100%	1.00
mild	4	4	80%	100%	0.89
none	4	3	100%	75%	0.86
Tổng	**15**	**14**			**Accuracy 93.3%**

Bảng 4.2: Kết quả kiểm thử end-to-end sàng lọc SXHD qua LLM (15 ca)

Phân tích ca sai (E-N01): Bệnh nhân mô tả *"đang ở Hà Nội quận Ba Đình, không có ai xung quanh bị sốt xuất huyết"* — LLM suy luận in_epidemic_area=True do Hà Nội là thành phố có nguy cơ dịch tễ cao, trong khi đáp án chuẩn là False (không có ổ dịch cục bộ). Đây là **false positive an toàn** về mặt lâm sàng — hệ thống cảnh báo thừa nhưng không bỏ sót ca bệnh. Ngưỡng xác nhận dịch tễ có thể được tinh chỉnh bằng cách bổ sung danh sách vùng dịch cụ thể vào knowledge base.

Nhận xét: Hệ thống đạt độ chính xác cao với các ca severe và warning (F1 = 1.00), là hai mức độ quan trọng nhất trong thực tế lâm sàng vì sai sót ở đây có thể gây nguy hiểm tính mạng.

4.1.2 Kiểm thử module phân tích CBC

Bảng 4.3 trình bày kết quả kiểm thử module dengue_cbc_predict với các bộ tham số đầu vào khác nhau:

Đầu vào (PLT / WBC / HCT)	**Kết quả**	**Giải thích**
PLT=85, WBC=3.2, ngày sốt 4	likely	PLT<100 + WBC<4, ngày nguy hiểm
PLT=45, WBC=2.8, HCT tăng 37%, AST=520	very_likely	PLT<50 + warning score cao
PLT=240, WBC=6.5, ngày sốt 2	unlikely	CBC bình thường, không có tiêu chí
PLT=30, WBC=2.0, AST=1200	very_likely	Suy gan nặng, PLT<50
PLT=72, WBC=6.8	possible	Chỉ 1 tiêu chí (PLT<100)
PLT=18, WBC=1.5, HCT tăng 25%	very_likely	PLT<20 + HCT tăng mạnh
PLT=150, WBC=3.8	possible	Chỉ WBC giảm nhẹ
PLT=95, WBC=4.2, HCT tăng 22%	likely	PLT<100 + HCT tăng ≥20%

Bảng 4.3: Kết quả kiểm thử module đánh giá nguy cơ SXHD từ CBC

4.1.3 Kiểm thử module Sinh hiệu và cảnh báo CDS

Module sinh hiệu được kiểm thử ở 2 tầng:

Tầng 1 — Kiểm thử ngưỡng cảnh báo (unit test). 28 test case được viết để verify 5 hàm interpretation (_interp_temp, _interp_hr, _interp_rr, _interp_spo2, _interp_bp), phủ đủ 5 mức N/H/HH/L/LL. Kết quả: **28/28 case pass**, các edge case như HA 145/80 (THA độ 1 do isolated systolic) và HA 130/105 (THA độ 2 do isolated diastolic) đều được phân loại đúng theo nguyên tắc "lấy mức nặng hơn" của BYT 3192.

Tầng 2 — Kiểm thử end-to-end trên Aidbox. 4 case thực tế được POST lên Aidbox, sau đó GET lại để verify roundtrip:

- Case bình thường: HA 120/80, mạch 72, nhiệt độ 36.8°C, SpO2 98% → alerts rỗng.
- Case warning: HA 170/102, mạch 105, nhiệt độ 38.9°C → 3 alerts severity "warning".
- Case critical: HA 80/50, mạch 135, nhiệt độ 39.6°C, SpO2 88% → 4 alerts severity "critical".
- Case API endpoint qua POST /api/vitals: HA 145/95, mạch 88, nhiệt độ 37.2°C → saved + 1 warning.

Tất cả 4 record được lưu thành công vào Aidbox với observation_id và đọc lại được qua fhir_get_observations(category="vital-signs"), xác nhận tính toàn vẹn của module.

4.1.4 Kiểm thử module Drug Allergy Check 4 lớp

Module được kiểm thử với BN Nguyễn Văn A (đã ghi nhận dị ứng Penicillin severe + dị ứng Tôm cua moderate trên Aidbox):

Thuốc test	Verdict	Cảnh báo
amoxicillin-500	block	BN dị ứng nhóm penicillin — thuốc kê CÙNG nhóm dược lý, KHÔNG được kê
cefuroxime-500	warn	BN dị ứng Penicillin — Cephalosporin có nguy cơ phản ứng chéo 2–4.8%
azithromycin-500	clear	Không liên quan (Macrolide)
paracetamol-500	clear	Không liên quan
ibuprofen-400	clear	NSAID — BN không có dị ứng NSAID

Bảng 4.4: 5 test case allergy check với BN có dị ứng Penicillin severe

5/5 case verdict chính xác. Phần suggested_alternatives cũng trả về đúng (Azithromycin, Clarithromycin, Erythromycin, Ciprofloxacin... cho trường hợp block Amoxicillin).

Kiểm thử 3-tier alert trên UI. Test scenario kê đơn cho BN dị ứng Penicillin:

1. Tier 1: mở modal Complete Visit → banner đỏ "△ DỊ ỨNG: Penicillin (Nặng)" hiển thị ngay đầu form. **PASS**.
2. Tier 2: chọn Amoxicillin từ dropdown → block đỏ hiện ngay dưới dropdown 200ms, kèm 4 nút thay thế. Click Azithromycin → block đỏ biến mất. **PASS**.
3. Tier 3: cố ý thêm Amoxicillin lại không nhập override, bấm Lưu → submit bị chặn, modal đỏ yêu cầu nhập lý do $\geq$5 ký tự. Nhập "BN test pen da âm tính 2025" → lưu thành công với note.text ghi rõ override reason. **PASS**.

4.1.5 Kiểm thử module Kê đơn và Email tóm tắt

Test 1 — Kê đơn thuốc thường (clear). Đơn 2 thuốc Azithromycin + Paracetamol cho BN có dị ứng Penicillin. Kết quả: saved=True, 2 MedicationRequest tạo trên Aidbox với cùng groupIdentifier.value, đọc lại qua fhir_get_prescriptions thấy đầy đủ. **PASS**.

Test 2 — Đơn có thuốc block (Amoxicillin). Đơn 1 thuốc Amoxicillin, không có override_reason. Kết quả: error: "prescription_blocked", không POST gì lên Aidbox. **PASS**.

Test 3 — Đơn có thuốc warn (Cefuroxime). Đơn 1 thuốc Cefuroxime (cross-react với Penicillin). Kết quả: saved=True, warnings[] có 1 entry, MedicationRequest được lưu. **PASS**.

Test 4 — Đơn có override. Đơn 1 thuốc Amoxicillin với override_reason = "BN test pen da âm tính 2025". Kết quả: saved=True, MedicationRequest có note[].text chứa override reason đầy đủ cho audit. **PASS**.

Test 5 — Email tóm tắt khám. Gọi fhir_send_visit_summary với 2 thuốc (Azithromycin + Paracetamol), email đích là Gmail test. Kết quả: email được gửi thành công qua Gmail SMTP TLS, render HTML đẹp trong Gmail/Outlook, có đầy đủ chẩn đoán, ghi chú, kế hoạch, bảng thuốc và chữ ký BS. **PASS**.

4.1.6 Kiểm thử phân quyền theo vai trò

Bảng 4.5 kiểm tra tính đúng đắn của phân quyền:

Thao tác	Role	Kết quả	Ghi chú
Phân tích CBC	patient	Bị chặn	Tool không có trong bộ
Phân tích CBC	doctor_cbc	Thành công	Có trong toolset
Duyệt lịch hẹn	patient	Bị chặn	Tool không tồn tại
Duyệt lịch hẹn	doctor_ appointments	Thành công	Có trong toolset
Xem hồ sơ BN	patient	Bị chặn	Không có quyền
Xem hồ sơ BN	doctor_records	Thành công	Có trong toolset
Đặt lịch	patient	Thành công	Có trong toolset bệnh nhân
Upload ảnh CBC	patient	Không có nút	UI ẩn nút upload
Xem hồ sơ của mình	patient (đã đăng nhập)	Thành công	Token xác thực đúng patient_id
Xem hồ sơ người khác	patient (đã đăng nhập)	Bị chặn	Agent từ chối theo system prompt
Giả mạo patient_id	patient (gửi thẳng API)	Bị chặn	Backend bỏ qua, dùng token

Bảng 4.5: Kiểm thử phân quyền theo vai trò

4.1.7 Kiểm thử luồng chức năng

Các luồng chính được kiểm thử end-to-end qua giao diện web:

Flow	Mô tả	Kết quả	Tool được gọi
A	Đặt lịch trực tiếp với khoa và ngày cụ thể	PASS	fhir_create_appointment
B	Triage từ triệu chứng, phân mức ưu tiên	PASS	fhir_triage_assessment
C	Gợi ý khoa từ mô tả triệu chứng	PASS	fhir_suggest_department (RAG)
D	Sàng lọc SXHD — bệnh nhân, 2 lượt	PASS	dengue_symptom_assessment
E	Phân tích CBC từ ảnh — bác sĩ	PASS	fhir_create_cbc + dengue_cbc_predict
F	Duyệt/từ chối lịch hẹn	PASS	fhir_confirm/reject_appointment
G	Hủy lịch hẹn bằng tên bệnh nhân	PASS	fhir_cancel_appointment
H	Xem lịch sử xét nghiệm	PASS	fhir_get_observations
I	Bệnh nhân đăng nhập xem hồ sơ của mình	PASS	fhir_get_patient_history (token auth)

Bảng 4.6: Kết quả kiểm thử các luồng chức năng chính

4.1.8 Kiểm thử module tra cứu số liệu dịch bệnh

Module được kiểm thử với các tuần cách xa nhau để xác nhận tính tổng quát của bộ regex, cùng các trường hợp biên (tuần một chữ số, tuần không tồn tại). Số liệu đối chiếu trực tiếp với bản tin gốc trên cổng HCDC.

Truy vấn	Kết quả mong đợi	Thực tế
Tuần mới nhất (qua category)	Lấy tuần 23, tự lấy tuần 22 để so sánh	PASS — 409 ca, +12 so tuần 22
week=22 (tra động qua search)	397 ca, so với tuần 21	PASS — +25 ca (+6,7%)
week=5 (tuần một chữ số)	Parse đúng tuần 5/2026	PASS — 1170 ca
week=99 (không tồn tại)	Từ chối, liệt kê tuần có sẵn	PASS — không bịa số
Hỏi lại tuần đã tra	Đọc cache, không gọi mạng	PASS — ≈ 2 ms
"so với tuần trước"	So sánh tính bằng code	PASS — khớp số học

Bảng 4.7: Kết quả kiểm thử module tra cứu số liệu dịch bệnh

Trường hợp week=5 xác nhận biểu thức (\d{1,2})(\d{4}) xử lý đúng URL tuần một chữ số (tuan-52026); trường hợp week=99 xác nhận cơ chế chống bịa số: khi không tìm

thấy bản tin, hệ thống trả thông báo kèm danh sách tuần đang có thay vì sinh ra con số không có thật.

4.1.9 Kiểm chứng độ chính xác trích dẫn của module tra cứu thuốc

Khác với các module có đáp án số học rõ ràng, chất lượng của drug_info nằm ở chỗ câu trả lời có **truy ngược được về nguồn** hay không. Phương pháp kiểm thử là: với mỗi câu trả lời, mở từng tài liệu được trích dẫn và đối chiếu xem phát biểu có thực sự xuất hiện trong nguồn đó không (fact-checking thủ công).

Ví dụ điển hình — truy vấn "tác dụng phụ của cefuroxime". Phát biểu "siêu nhiễm *Ps. aeruginosa*/enterococci" được đối chiếu và tìm thấy **đúng nguyên văn** trong abstract: "*Superinfections with Ps. aeruginosa and enterococcal strains may present a problem*" — nghĩa là nội dung không bịa. Tuy nhiên ở phiên bản gộp ngữ cảnh, số trích dẫn lại trỏ sai nguồn (Bảng 3.3). Sau khi áp dụng cơ chế gắn nhãn [n] cho từng đoạn bằng chứng, mọi chip trích dẫn đều trỏ đúng tài liệu chứa phát biểu.

Tiêu chí kiểm thử	**Kết quả**
Nội dung phát biểu có trong nguồn trích dẫn	PASS — đối chiếu khớp abstract
Khi nguồn không nêu liều, không tự bịa số mg	PASS — chỉ tới chuyên luận StatPearls
Số trích dẫn [n] trỏ đúng tài liệu	PASS (sau khi gắn nhãn nguồn)
Không gán nhãn nguồn quốc tế là "Việt Nam"	PASS
Mở rộng từ đồng nghĩa (*paracetamol*/*acetaminophen*)	PASS — có kết quả cho cả hai

Bảng 4.8: Kết quả kiểm chứng grounding và độ chính xác trích dẫn của drug_info

Kết quả cho thấy ràng buộc grounding kết hợp gắn nhãn nguồn giúp module **trung thực và kiểm chứng được**: thông tin có trong nguồn được dẫn đúng, thông tin không có (như liều cụ thể trong abstract) được chủ động chỉ tới chuyên luận thay vì bịa. Hạn chế còn lại là nguồn tiếng Anh quốc tế, chưa theo chuẩn kê đơn Việt Nam.

4.1.10 Đo hiệu năng thời gian phản hồi

Bảng 4.9 đo thời gian phản hồi trung bình (trên 5 lần chạy) trong môi trường localhost: Intel Core i5, RAM 8 GB, Aidbox Docker cục bộ, Qwen3-VL-Plus qua Alibaba DashScope (RTT ≈ 150 ms). Thời gian được tính từ lúc gửi request đến khi nhận token phản hồi cuối cùng qua SSE.

Flow	Mô tả	Số tool call	Thời gian TB (s)	Ghi chú
A	Đặt lịch khám	3	4.8	suggest_dept + get_slot + create_appt
B	Triage thông minh	2	2.5	triage_assessment + suggest_dept
C	Gợi ý khoa (RAG)	1	1.5	Chỉ suggest_dept (cosine similarity)
D (lượt 1)	Sàng lọc SXHD — hỏi bổ sung	0	0.8	Không gọi tool, chỉ text generation
D (lượt 2)	Sàng lọc SXHD — trả kết quả	1	2.1	dengue_symptom_ assessment
E	Phân tích CBC từ ảnh	3	4.5	Vision + search_patient + create_cbc + cbc_predict
F	Duyệt/từ chối lịch hẹn	2	2.3	get_pending + confirm/reject
G	Hủy lịch hẹn	2	2.6	search_patient + cancel_appointment
H	Xem kết quả xét nghiệm	2	1.9	search_patient + get_observations

Bảng 4.9: Thời gian phản hồi trung bình theo luồng (đo trên môi trường localhost)

Một số nhận xét từ kết quả đo:

- **Flow D lượt 1** nhanh nhất (0.8 s) do không gọi tool nào, LLM chỉ đặt câu hỏi làm rõ triệu chứng.
- **Flow E** chậm nhất (4.5 s) vì phải xử lý vision (encode ảnh + inference) cộng thêm 3 tool calls tuần tự.
- **Flow A** (4.8 s) có 3 tool calls nhưng không có vision; thời gian chủ yếu từ 2 lần gọi Aidbox REST API và 1 lần gọi LLM xác nhận trước khi tạo Appointment.
- Các flow chỉ có 1–2 tool calls đạt thời gian phản hồi dưới 2.6 s, phù hợp với yêu cầu phi chức năng đề ra.

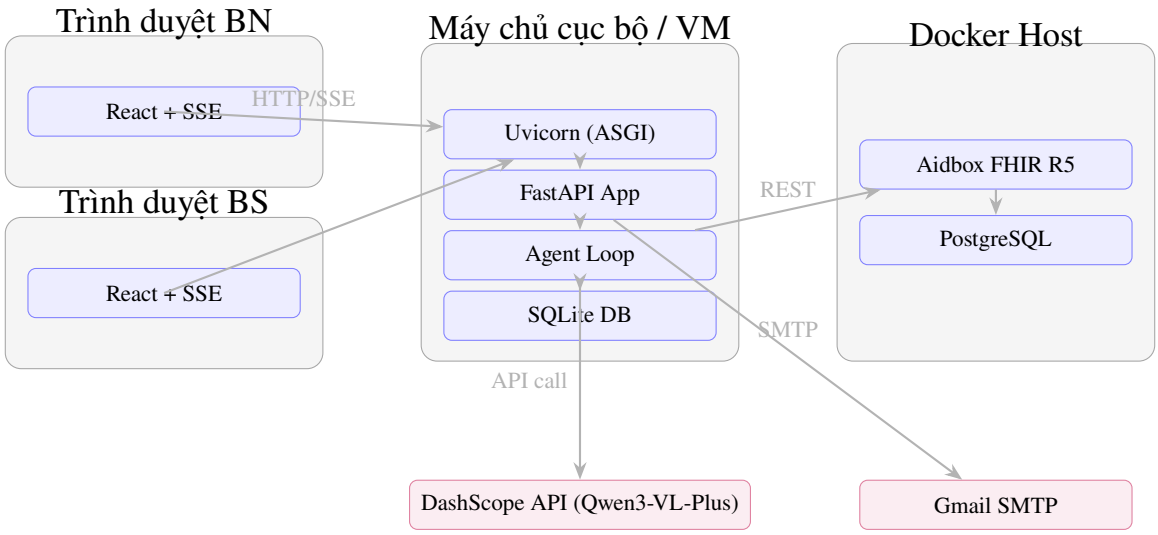
4.2 Triển khai hệ thống

```
# Chay Aidbox FHIR
docker run --name aidbox -e AIDBOX_CLIENT_ID=root -
e AIDBOX_CLIENT_SECRET=secret -p 8888:8888 healthsamurai/aidboxone:l

# Cau hinh bien moi truong
export AIDBOX_URL=http://localhost:8888
export ANTHROPIC_API_KEY=<key>

# Khoi dong HIS Agent
python main.py  # Chay tai http://localhost:8000
```

Đoạn mã: Khởi động hệ thống.



Hình 4.1: Sơ đồ triển khai hệ thống HIS Agent

Kết luận

Kết quả của đồ án

1. Trình bày và hệ thống hóa các kiến thức nền tảng về kiến trúc agent, chuẩn HL7 FHIR R5, kỹ thuật RAG, và ứng dụng LLM trong lĩnh vực y tế số.
2. Thiết kế và triển khai pipeline agent với 26 công cụ FHIR và 2 công cụ sàng lọc SXHD, bao phủ luồng khám lâm sàng hoàn chỉnh: đặt lịch, triage, đo sinh hiệu với cảnh báo CDS, ghi nhận dị ứng, kê đơn thuốc với kiểm tra dị ứng 3 lớp, và hoàn tất khám với email tóm tắt tự động.
3. Xây dựng module sàng lọc sốt xuất huyết Dengue theo QĐ 2760/QĐ-BYT 2023 và WHO 2009, với trích dẫn nguồn minh bạch và disclaimer phù hợp.
4. Triển khai đầy đủ hệ thống hoàn chỉnh: FastAPI SSE streaming, xác thực người dùng, phân quyền nghiêm ngặt, và giao diện web thời gian thực.

Kỹ năng đạt được

1. Hiểu và vận dụng được kiến trúc agent (tool-calling loop) trong thực tế.
2. Làm việc với chuẩn HL7 FHIR R5 và triển khai FHIR server.
3. Tích hợp LLM với vision API để xử lý ảnh y tế.
4. Xây dựng hệ thống streaming SSE với FastAPI và asyncio.
5. Chế bản đồ án bằng LaTeX.

Hướng phát triển trong tương lai

1. **LLM tiếng Việt on-premise**: Tích hợp mô hình LLM chạy cục bộ để bảo mật dữ liệu y tế.
2. **Mở rộng sàng lọc**: Bổ sung các bệnh lý khác (COVID-19, sốt rét, viêm phổi).
3. **Multi-agent**: Triển khai nhiều agent chuyên biệt (triage, scheduling, clinical support).
4. **Fine-tuning**: Nghiên cứu fine-tuning LLM trên dữ liệu lâm sàng tiếng Việt.
5. **Ứng dụng di động**: Phát triển app React Native/Flutter với giao diện thân thiện hơn.
6. **Kiểm định lâm sàng**: Tích hợp với HIS thực tế và đánh giá trong môi trường bệnh viện.

TÀI LIỆU THAM KHẢO

Tiếng Việt

[1] Bộ Y tế Việt Nam (2023), *Quyết định 2760/QĐ-BYT ngày 04/7/2023 về việc ban hành hướng dẫn chẩn đoán, điều trị sốt xuất huyết Dengue*. Hà Nội: Bộ Y tế.

[2] Nguyễn Đình Hòa (2020), *Thực trạng và giải pháp ứng dụng Hệ thống thông tin bệnh viện tại Việt Nam*. Tạp chí Y học Việt Nam, 490(1), 45–52.

[3] Bệnh viện Bạch Mai (2022), *Hệ thống Thông tin Bệnh viện HIS-EMR – Giới thiệu và triển khai tại Bệnh viện Bạch Mai*. `https://bachmai.gov.vn`

[4] VietSoftware International (2023), *eHospital – Giải pháp Quản lý Bệnh viện Toàn diện*. `https://vietsoftware.com/ehospital`

[5] Công ty Cổ phần Giải pháp Y tế Việt Nam – VMED (2023), *Medisoft HIS – Phần mềm Quản lý Bệnh viện*. `https://vmed.vn`

Tiếng Anh

[6] World Health Organization (2009), *Dengue: Guidelines for Diagnosis, Treatment, Prevention and Control*. Geneva: WHO Press. ISBN 978-92-4-154787-1.

[7] HL7 International (2023), *HL7 FHIR Release 5*. `https://hl7.org/fhir/R5/`

[8] OpenAI (2023), *Function Calling and other API Updates*. `https://openai.com/blog/function-calling-and-other-api-updates`

[9] Anthropic (2024), *Claude Haiku 4.5 Technical Report*. `https://www.anthropic.com/claude`

[10] Health Samurai / Medplum (2024), *Aidbox: FHIR-native Healthcare Platform*. `https://www.health-samurai.io/aidbox`

[11] Reimers, N., & Gurevych, I. (2019), *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Proceedings of EMNLP 2019.

[12] Lewis, P., et al. (2020), *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.

[13] Ramírez, S. (2019), *FastAPI*. `https://fastapi.tiangolo.com`

[14] Singhal, K., et al. (2023), *Towards Expert-Level Medical Question Answering with Large Language Models*. arXiv:2305.09617.

[15] W3C (2021), *Server-Sent Events – W3C Recommendation*. https://www.w3.org/TR/eventsource/

[16] Hsieh, T. C., et al. (2004), *Characteristics and consequences of drug allergy alert overrides in a computerized physician order entry system*. American Journal of Health-System Pharmacy (AJHP), 61(8).

[17] Bates, D. W., et al. (2001), *Reducing the frequency of errors in medicine using information technology*. Journal of the American Medical Informatics Association (JAMIA), 8(4), 299–308.

[18] Preston, C. L. (ed.) (2022), *Stockley's Drug Interactions*, 13th ed. London: Pharmaceutical Press.

[19] StatPearls (NCBI Bookshelf), *Cephalosporins (NBK459320)* và các chuyên luận thuốc liên quan. https://www.ncbi.nlm.nih.gov/books/NBK459320/

[20] Bộ Y tế Việt Nam (2010), *Quyết định 3192/QĐ-BYT về Hướng dẫn chẩn đoán và điều trị tăng huyết áp*. Hà Nội: Bộ Y tế.

[21] Royal College of Physicians (2017), *National Early Warning Score (NEWS) 2: Standardising the assessment of acute-illness severity in the NHS*. London: RCP.

[22] Loscalzo, J., et al. (eds.) (2022), *Harrison's Principles of Internal Medicine*, 21st ed. New York: McGraw-Hill.

[23] Bộ Y tế Việt Nam (2022), *Quyết định 1165/QĐ-BYT về Danh mục thuốc thiết yếu*. Hà Nội: Bộ Y tế.

[24] Bộ Y tế Việt Nam (2013), *Quyết định 1088/QĐ-BYT về Hướng dẫn theo dõi phản ứng có hại của thuốc*. Hà Nội: Bộ Y tế.